NSCT Preparation
File 05: Software Engineering (10%)

This file follows the official P@SHA syllabus. It emphasizes real-world application, professional ethics, SDLC comparisons, and modern agile practices. Each question provides the correct answer in **bold** followed by a deep-dive explanation.

Master Table of Contents (File 05 - Target: 180+ MCQs)

#	Official Subtopic	Coverage	MCQs
01	Introduction to Software Engineering	Basic	11
02	Software Process Models	Standard	15
03	Agile Software Development	Extended	20
04	Software Requirements Engineering	Standard	15
05	Software Project Management	Standard	15
06	Software Design	Standard	15
07	Software Architecture	Standard	15
08	User Interface Design	Basic	10
09	Software Implementation & Coding	Basic	10
10	Software Testing	Extended	20
11	Software Maintenance & Evolution	Basic	10
12	Software Quality Assurance	Basic	10
13	Software Metrics & Measurement	Basic	10
14	Software Configuration Management	Basic	10
15	Software Risk Management	Basic	10

16	Software Security Engineering	Basic	10
Total MCQs (File 05)			**206**

Subtopic 01: Introduction to Software Engineering (11 MCQs)

Quick Notes

- **Software Engineering (SE):** An engineering discipline concerned with all aspects of software production from initial stages to maintenance.
- **The Crisis:** The "Software Crisis" refers to the difficulty of writing correct, understandable, and verifiable computer code as systems scale.
- **Dual Nature:** Software is both a *product* (transforms information) and a *vehicle* (delivers the product).
- **Attributes of Good Software:** Maintainability, Dependability, Efficiency, and Acceptability (UI/UX).
- **Ethical Responsibility:** SE is not just about code; it includes professional integrity, privacy, and respecting intellectual property.

Common Traps & Confusions

- **SE vs. Computer Science:** CS focuses on theory and fundamentals; SE focuses on the practicalities of developing and delivering useful software in a cost-effective way.
- **Program vs. Software Product:** A program is just code. A "Software Product" includes documentation, configuration, and data required to make the program useful.
- **Myth of "Adding Manpower":** Brooks’s Law states that adding manpower to a late software project makes it even later.

MCQs

Q1. (Scenario-Based) A talented student writes a 500-line script to automate their homework. A year later, a tech company builds a 500,000-line healthcare system. What is the fundamental difference that justifies the company using **Software Engineering** rather than just "coding"?

A) The student used Python, but the company must use C++.

B) The student's code is small enough to be perfect.

C) SE provides disciplined, systematic methods to manage the massive scale, complexity, and long-term maintenance of industrial systems.

D) Coding is for individuals; SE is only for groups of more than 50 people.

> Software Engineering is specifically designed to handle "Software in the large." While a script is a personal tool, industrial software must be maintainable, scalable, and reliable across years of use by different teams.

Q2. (Comparison) How does **Software Engineering** differ from **Systems Engineering**?

A) Systems engineering is only for hardware; SE is only for code.

B) Systems engineering deals with all aspects of computer-based systems (hardware, software, and processes); SE is a subset focusing on the software component.

C) There is no difference; they are synonymous.

D) Software engineering is theoretical, while systems engineering is practical.

> Systems engineering is the high-level umbrella that includes hardware design, software development, and the human/business processes that surround the technology. SE focuses strictly on the software parts of that larger system.

Q3. (Conceptual) Which of the following is considered an **"Essential Attribute"** of high-quality professional software?

A) Using the most recent version of a programming language.

B) Including at least 50% comments in the source code.

C) Maintainability (the ability of the software to evolve to meet changing needs).

D) The ability to run on obsolete hardware from 20 years ago.

> Maintainability is vital because software costs are often higher *after* release. If software cannot be updated to fix bugs or add features without breaking, it is considered low quality.

Q4. (Scenario-Based) A software manager believes that if they fall behind schedule, they can simply hire three more developers to "catch up." This is a violation of which famous software engineering principle?

A) The Pareto Principle

B) Brooks's Law

C) Moore's Law

D) The Law of Diminishing Returns

> Brooks's Law states: "Adding manpower to a late software project makes it later." This is because new developers need training from the existing ones, and the communication overhead increases exponentially.

Q5. (Comparison) Compare "Generic Software Products" with "Customized Software Products." What defines the **Customized** category?

A) It is sold on the open market to any customer who can buy it.

B) It is commissioned by a specific customer and developed to meet their unique business requirements.

C) It is always cheaper than generic software.

D) It does not require a maintenance phase.

> Generic products (like MS Word) are built for the general market. Customized products (like a specific bank's internal ledger system) are built specifically for one client's needs.

Q6. (Code/Console Logic) A junior developer is setting up their first professional project. They use the following command. In the context of "Introduction to SE," why is this simple action considered a move toward "Engineering" rather than "Amateur Coding"?

bash - terminal

```bash
$ git init
$ touch README.md CONTRIBUTING.md
```

A) It makes the code run faster.

B) It establishes version control and documentation frameworks, which are essential for collaboration and maintainability.

C) git init is a requirement for the compiler to work.

D) It automatically writes the software requirements.

> Software Engineering emphasizes processes. Using Version Control (Git) and creating documentation (README) allows the project to be managed as an engineering asset rather than a loose collection of files.

Q7. (Conceptual) What is the primary cause of the "Software Crisis" that led to the birth of Software Engineering as a discipline in the late 1960s?

A) Computers were not powerful enough to run software.

B) There were no programming languages available.

C) The complexity of software projects grew faster than the ability of developers to manage and maintain them, leading to massive overruns.

D) The high cost of hard drives.

> As hardware became more powerful, software became more ambitious. Without formal engineering methods, projects were consistently over budget, late, and full of bugs—this was the "Crisis."

Q8. (Scenario-Based) An engineer is asked to work on a medical device where a software bug could result in the death of a patient. Which attribute of software is the **absolute highest priority** in this scenario?

A) Efficiency (low memory usage)

B) Acceptability (pretty UI)

C) Dependability (Safety and Reliability)

D) Maintainability

> Dependability includes reliability, security, and safety. In safety-critical systems, the software must function correctly under all circumstances to prevent loss of life.

Q9. (Comparison) What is the difference between **Essential Complexity** and **Accidental Complexity** in software engineering?

B) Essential complexity is inherent in the problem being solved; Accidental complexity is caused by poor choices in tools, language, or design.

A) Essential is for backend; Accidental is for frontend.

C) Accidental complexity is required for security; Essential is not.

D) Essential complexity can be removed; Accidental cannot.

> Essential complexity is the "hard part" of the logic (e.g., calculating taxes). Accidental complexity is the difficulty we create for ourselves by using a bad database or a messy programming style.

Q10. (Conceptual) According to the IEEE definition, Software Engineering is the application of a systematic, disciplined, _____ approach to the development, operation, and maintenance of software.

A) Randomized

B) Emotional

C) Quantifiable

D) Theoretical

> The official definition emphasizes that SE is "Systematic, Disciplined, and Quantifiable." This means we measure things like bug counts, performance, and costs.

Q11. (Scenario-Based) A developer discovers a critical security flaw in a competitor's software. Instead of reporting it, they post it on a public forum to embarrass the competitor. This action is a violation of:

A) Software Efficiency Protocols

B) Professional Ethics and the ACM/IEEE Code of Ethics

C) Software Configuration Management

D) The V-Model

> Professional software engineers have an ethical obligation to act in the public interest and maintain the integrity of the profession. Maliciously exposing flaws violates professional conduct.

Deep Dive: The Mythical Man-Month

Fred Brooks, the manager of the OS/360 project at IBM, wrote "The Mythical Man-Month," which is considered one of the founding texts of SE. His key insights include:

- **The Man-Month is a Myth:** Estimating a project as "12 man-months" implies that 1 person working for 12 months is the same as 12 people working for 1 month. This is false because software tasks are *sequential* and *interdependent*.
- **Communication Overhead:** If *n* people work on a project, there are `n(n-1)/2` communication channels. As *n* grows, the time spent talking about work exceeds the time spent doing work.
- **Plan to Throw One Away:** The first version of a software product is often a "pilot." You will likely need to rewrite it once you truly understand the requirements.

Stable Web Resources

- Wikipedia: Software Engineering
- IEEE: Code of Ethics

Quick Revision Cheat Sheet

- **SE Goals:** On time, within budget, high quality.
- **Brooks's Law:** Adding people to a late project makes it later.
- **Essential Attributes:** Maintainable, Dependable, Efficient, Acceptable.
- **Software =** Programs + Documentation + Configuration + Data.
- **CS vs SE:** CS is theory; SE is the practice of delivering products.

END OF SUBTOPIC 01 — MOVING TO SUBTOPIC 02

Subtopic 02: Software Process Models (15 MCQs)

Quick Notes

- **Software Process:** A structured set of activities required to develop a software system (Specification, Design, Evolution, Validation).
- **Waterfall Model:** Linear-sequential. Phases do not overlap. Best for stable, well-understood requirements.
- **V-Model:** An extension of Waterfall that emphasizes verification and validation. Every development phase has a corresponding testing phase.
- **Incremental Model:** The product is broken into small chunks (increments) and delivered piece by piece.
- **Iterative Model:** The entire system is built and then refined in cycles. Each version improves upon the last.
- **Spiral Model:** An evolutionary model that focuses heavily on **Risk Analysis**. It combines iterative development with the systematic aspects of Waterfall.

Common Traps & Confusions

- **Incremental vs. Iterative:** Incremental adds "more features" over time; Iterative adds "more detail/quality" to existing features.
- **Waterfall "Backtracking":** In theory, Waterfall has no backtracking. If a mistake is found in a late stage, the whole process must technically restart or enter an expensive "change request" loop.
- **V-Model is NOT Agile:** Even though it mentions testing early, it is still a rigid, plan-driven model similar to Waterfall.

MCQs

Q1. (Scenario-Based) A government defense agency is commissioning a flight control system. The requirements are extremely rigid, legally binding, and unlikely to change during development. Failure could result in loss of life. Which process model is most appropriate?

A) Incremental Development

B) Evolutionary Prototyping

C) Waterfall Model

D) Big Bang Model

> While Waterfall is often criticized for being slow, its rigid structure is ideal for high-security, safety-critical systems with fixed requirements where massive documentation and traceability are legally required.

Q2. (Comparison) How does the **Incremental Model** fundamentally differ from the **Waterfall Model** in terms of delivery?

A) Incremental requires no testing phase.

B) Waterfall delivers the software in multiple parts.

C) Incremental allows the customer to receive and use core functionality early, while Waterfall only delivers the final product at the end of the lifecycle.

D) Waterfall is faster for large-scale projects.

> In Waterfall, you get nothing until the very end. In Incremental development, the system is split into "increments," and the most important features (the core) are delivered first, providing value to the user earlier.

Q3. (Scenario-Based) A medical tech startup is building a new heart-rate monitoring app. The founder has a vague idea of the UI but isn't sure how users will react to the features. They decide to build a "quick and dirty" version to show users and gather feedback. This is known as:

A) The V-Model

B) Prototyping

C) Formal Methods

D) Component-Based Software Engineering

> Prototyping is used when requirements are unclear. It allows the developer to build a simplified version of the software to clarify requirements and ensure the final product meets user expectations.

Q4. (Conceptual) In the **V-Model** of software development, which testing phase is directly mapped to the **Requirements Analysis** phase?

A) Unit Testing

B) Integration Testing

C) System Testing

D) Acceptance Testing

> The V-Model maps development steps to testing steps. Requirements Analysis (what the user wants) is validated during Acceptance Testing (checking if the user is happy with the final product).

Q5. (Scenario-Based) An international bank is developing a multi-million dollar global payment gateway. The project involves massive technical uncertainty and potential financial risks. Which model was specifically designed to handle such **Risk-Heavy** environments?

A) Waterfall Model

B) Iterative Model

C) Spiral Model

D) V-Model

> The Spiral Model, developed by Barry Boehm, is unique because its primary driver is "Risk." Every "orbit" of the spiral begins with risk analysis to determine if the project should continue or be abandoned.

Q6. (Comparison) What is the primary disadvantage of the **Waterfall Model** when used for modern web applications?

A) It is difficult to accommodate changing requirements once the development phase has started.

B) It requires too much high-end hardware.

C) It does not include a design phase.

D) It is only compatible with C++.

> Modern web apps evolve rapidly. The Waterfall model's "frozen" requirement phase makes it nearly impossible to adapt to user feedback or market changes midway through the project.

Q7. (Code/Console Logic) A DevOps engineer is looking at a pipeline configuration. Which software process concept is most closely represented by this automated "Code -> Build -> Test -> Deploy" loop?

yaml - pipeline_config

```yaml
stages:
  - build
  - test
  - deploy_to_staging
  - deploy_to_production
```

A) Requirement Engineering

B) The Software Lifecycle (SDLC) as a repeatable process

C) Waterfall Documentation

D) Risk Analysis

> Modern DevOps practices automate the Software Development Life Cycle (SDLC). The stages in the YAML file represent the transition of a product through the "Process," emphasizing continuous validation and delivery.

Q8. (Conceptual) Which phase of the generic software process focuses on ensuring the software meets the user's needs and satisfies the specification?

A) Specification

B) Development

C) Validation

D) Evolution

> Validation (Testing) is the phase where we ask: "Are we building the right product?" and "Is the product building correctly?" to ensure it meets requirements.

Q9. (Scenario-Based) A team is building a software system for a client who is available every day to give feedback. The team decides to deliver a working version every 2 weeks to get the client's input. This is an example of:

A) Waterfall Development

B) Formal Development

C) Evolutionary/Incremental Development

D) Systems Engineering

> Evolutionary development (the foundation of Agile) involves creating a series of versions (increments) and using feedback to refine the next version.

Q10. (Comparison) In the **V-Model**, which testing level is focused on checking if individual software components/modules work correctly in isolation?

A) Unit Testing

B) Integration Testing

C) System Testing

D) Beta Testing

> Unit Testing is the lowest level of testing, corresponding to the "Coding" or "Detailed Design" phase, where individual functions or classes are tested.

Q11. (Conceptual) What does the **Horizontal Axis** of the Spiral Model represent?

A) Time spent on the project.

B) Number of developers hired.

C) Cumulative cost incurred in the project.

D) The number of bugs found.

> In the Spiral Model diagram, the distance from the origin (the radius) represents the cumulative cost of the project as it goes through more iterations.

Q12. (Scenario-Based) You are a project manager. You realize that your project requirements are 80% similar to a previous project. You decide to assemble the system using existing modules and libraries. This is known as:

A) Waterfall Model

B) Spiral Model

C) Component-Based Software Engineering (CBSE)

D) Cleanroom Development

> CBSE focuses on software reuse. Instead of writing code from scratch, the process is centered on integrating existing software "components" into a new system.

Q13. (Comparison) Why is **Software Evolution** (Maintenance) often considered the most expensive phase of the software process?

A) Because computers become more expensive over time.

B) Because software exists for many years, and the cost of changing existing code, fixing bugs, and adapting to new hardware exceeds the initial development cost.

C) Because programmers get paid more as they get older.

D) Because documentation is expensive to print.

> In the industrial world, 60% to 90% of software costs occur after the initial version is released. Evolution involves keeping the software useful as the world changes around it.

Q14. (Conceptual) In the **Spiral Model**, what activity occurs in the **Second Quadrant** (Top Left) of every cycle?

A) Planning the next phase.

B) Evaluation of alternatives and Identification/Resolution of Risks.

C) Coding and testing.

D) Delivering the product to the customer.

> Quadrant 2 of the Spiral is the Risk Analysis phase. This is the "soul" of the model, where the team decides if the technical or financial risks are too high to proceed.

Q15. (Scenario-Based) A company is developing a simple "Calculator" app. The requirements are 100% fixed, the technology is simple, and the project duration is only 2 weeks. Which model is most efficient here?

A) Waterfall Model

B) Spiral Model

C) Incremental Model

D) Agile/Scrum

> For very small, short-term projects with zero requirement volatility, the overhead of Agile or the risk analysis of Spiral is unnecessary. Waterfall is the most direct path to completion.

Deep Dive: The Cost of Change Curve

A fundamental driver of software process choice is the **Cost of Change Curve**.

- In **Waterfall**, the cost of change increases *exponentially* as the project progresses. A requirement change in the "Testing" phase might cost 100x more than a change in the "Requirement" phase because it forces rework across design, code, and documentation.
- In **Iterative/Agile** models, the goal is to keep the cost of change curve *flat* through automated testing, modular architecture, and continuous integration. This allows teams to embrace change even late in the development cycle.

Stable Web Resources

- Wikipedia: Software Development Process
- GeeksforGeeks: Software Process Framework

Quick Revision Cheat Sheet

- **Waterfall:** Linear, Plan-driven, No backtracking. Best for stable projects.
- **V-Model:** Emphasizes Validation/Verification at every step.
- **Incremental:** Deliver core first, then add more "What".
- **Iterative:** Build full system, then improve "How".
- **Spiral:** Risk-driven. Multi-cycle. Very expensive and complex.
- **Prototype:** Clarifies requirements through "throwaway" builds.

END OF SUBTOPIC 02 — MOVING TO SUBTOPIC 03

Subtopic 03: Agile Software Development (20 MCQs)

Quick Notes

- **Agile Manifesto:** Prioritizes Individuals/Interactions over processes, Working Software over documentation, Customer Collaboration over negotiation, and Responding to Change over following a plan.
- **Scrum Roles:**
  - *Product Owner:* Defines the "What" (Backlog).
  - *Scrum Master:* Process facilitator (removes blockers).
  - *Development Team:* Self-organizing (does the work).
- **Scrum Ceremonies:** Sprint Planning, Daily Stand-up (15 min), Sprint Review (Demo), Sprint Retrospective (Process improvement).
- **XP (Extreme Programming):** Focuses on technical excellence. Practices include Pair Programming, Test-Driven Development (TDD), and Continuous Integration.
- **Kanban:** Focuses on "Visualizing Work" and "Limiting Work in Progress (WIP)" to optimize flow.

Common Traps & Confusions

- **Scrum Master is NOT a Project Manager:** A Scrum Master does not assign tasks or give orders; they lead by serving and coaching.
- **"Agile means NO documentation":** False. Agile values "working software" *more*, but still requires necessary documentation.
- **Sprint Review vs. Retrospective:** The Review is about the *Product* (demoing features). The Retrospective is about the *Team/Process* (how we worked).

MCQs

Q1. (Scenario-Based) A customer realizes midway through a development cycle that a new government regulation requires an extra data-privacy field in the user profile. In a strictly Agile environment, how should the team handle this?

A) Refuse the change because the requirements were "frozen" at the start.

B) Charge the customer a massive "change request" fee and stop development.

C) Embrace the change by adding it to the Product Backlog and prioritizing it for a future Sprint.

D) Re-start the entire project from the requirements phase.

> One of the core pillars of the Agile Manifesto is "Responding to Change over following a plan." Agile processes are designed to handle requirement volatility gracefully through iterative prioritization.

Q2. (Comparison) In a Scrum team, who is primarily responsible for maintaining the **Product Backlog** and ensuring the team works on the most valuable features first?

A) Scrum Master

B) Product Owner

C) Lead Architect

D) Project Manager

> The Product Owner is the "voice of the customer." Their primary role is to maximize the value of the product by managing and prioritizing the backlog items.

Q3. (Scenario-Based) During a Daily Stand-up, a developer mentions that they are stuck because the database server keeps crashing, preventing them from testing their code. What is the **Scrum Master's** immediate responsibility?

A) Tell the developer to work harder and fix the server themselves.

B) Assign the task of fixing the server to another developer.

C) Take ownership of removing this "impediment" so the team can resume work.

D) Cancel the Sprint because the environment is unstable.

> A Scrum Master is a "Servant Leader." One of their primary duties is to shield the team from outside interference and remove any blockers (impediments) that slow down the development progress.

Q4. (Conceptual) Which Agile technical practice involves two programmers working at a single workstation—one writing code (the Driver) and the other reviewing each line as it is typed (the Navigator)?

A) Test-Driven Development (TDD)

B) Continuous Deployment

C) Pair Programming

D) Code Refactoring

> Pair Programming is a key practice in Extreme Programming (XP). It improves code quality, reduces bugs, and facilitates knowledge sharing within the team.

Q5. (Calculation/Logic) A team has a **Velocity** of 30 Story Points per Sprint. The Product Backlog has a total of 150 Story Points. Approximately how many Sprints will it take to complete the project?

A) 3 Sprints

B) 5 Sprints

C) 10 Sprints

D) 30 Sprints

> Velocity is a measure of how much work a team can handle in a single Sprint. Total Points (150) / Velocity (30) = 5 Sprints.

Q6. (Code/Console Logic) A developer is practicing **TDD (Test-Driven Development)**. They just wrote a test case and executed the following command. What is the expected result if they are following the TDD cycle correctly?

bash - unit_tests

```bash
$ npm test
```

A) All tests should pass (Green).

B) The new test should fail (Red) because the logic hasn't been written yet.

C) The compiler should throw a syntax error.

D) The computer should restart.

> The TDD cycle is "Red-Green-Refactor." You first write a test for a feature that doesn't exist yet, so the test *must* fail (Red). Then you write just enough code to make it pass (Green).

Q7. (Comparison) What is the main difference between a **Sprint Review** and a **Sprint Retrospective**?

A) The Review is for stakeholders to see the product features; the Retrospective is for the team to discuss internal process improvements.
B) The Review happens daily; the Retrospective happens once per year.
C) There is no difference; they are the same meeting.
D) The Review is for the Scrum Master; the Retrospective is for the Product Owner.

> Review = "What" we built (Product focus). Retrospective = "How" we worked (Team focus). Both are essential for the Agile pillar of "Inspect and Adapt."

Q8. (Scenario-Based) A team is using a **Kanban** board. The "Testing" column has a "Work in Progress (WIP) Limit" of 3. There are currently 3 tasks in the Testing column. A developer finishes a coding task and wants to move it to Testing. What should happen?

A) Move the task anyway; limits are just suggestions.
B) The task must wait in the "Ready for Testing" queue because the limit has been reached.
C) Delete one of the tasks in testing to make room.
D) Fire the tester for being slow.

> The core of Kanban is limiting WIP. If a column is at its limit, no new work can enter. This highlights bottlenecks and forces the team to help clear the Testing column before starting new work.

Q9. (Conceptual) In Agile estimation, what is the primary purpose of using **"Story Points"** instead of "Man-Hours"?

A) Hours are too difficult to calculate.
B) To make the team work longer hours.
C) To estimate the relative effort, complexity, and risk of a task rather than the exact time, which varies by developer.
D) Story points are required by the C++ compiler.

> Time estimation is notoriously inaccurate in software. Story points allow teams to estimate based on relative difficulty (e.g., "This task is twice as hard as that one"), which leads to more stable velocity over time.

Q10. (Scenario-Based) An Agile team is halfway through a 2-week Sprint. The CEO enters the room and demands that the team immediately stop what they are doing and build a new login screen for a meeting in 2 hours. How should the **Scrum Master** respond?

A) Apologize and immediately assign the task to the best developer.

B) Politely direct the CEO to the Product Owner to discuss the change for a future Sprint, protecting the team's current commitment.

C) Quit the job because the CEO is violating Agile rules.

D) Ask the team to work through the night to do both.

> Once a Sprint begins, the Sprint Goal and the work committed should not be changed by outside parties. The Scrum Master must protect the team from "Scope Creep" to ensure they can deliver their commitment.

Q11. (Comparison) Which Agile framework is most suitable for a "Maintenance" team that handles unpredictable incoming bug reports rather than a fixed set of features?

A) Scrum

B) Kanban

C) Waterfall

D) Spiral

> Scrum relies on fixed-length Sprints and planning. Kanban is "flow-based" and doesn't require Sprints, making it perfect for production support or maintenance where work arrives randomly.

Q12. (Conceptual) What does the "Definition of Done" (DoD) ensure in a Scrum environment?

A) That the Product Owner has paid the invoice.

B) That the code has been written.

C) That every increment meets the required quality standards (tests passed, code reviewed, documented) before it is considered "Finished."

D) That the office lights are turned off at 5 PM.

> The DoD is a shared understanding within the Scrum Team of what it means for work to be complete. Without a clear DoD, "Done" might mean different things to different people, leading to technical debt.

Q13. (Scenario-Based) A developer is writing a User Story. They use the format: "As a [User], I want [Feature] so that [Benefit]." Why is the **"So that [Benefit]"** part considered the most important?

A) It makes the story longer and more professional.

B) It provides the business context and value, helping the team understand *why* they are building the feature.

C) It is a syntax requirement for Jira software.

D) It lists the technical libraries required.

> In Agile, we focus on value. Understanding the "Why" allows developers to suggest better technical solutions that might achieve the same benefit more efficiently.

Q14. (Comparison) Compare **Continuous Integration (CI)** and **Continuous Delivery (CD)**. What is the specific goal of CI?

A) To deliver software to the customer every hour.

B) To ensure that code from different developers is frequently merged and tested to detect integration errors early.

C) To write the user manual automatically.

D) To replace the need for unit testing.

> CI is about merging code into a shared repository multiple times a day. Automated builds and tests run on every merge to ensure that the new code doesn't break existing functionality.

Q15. (Conceptual) What is a **"Sprint Burndown Chart"** used to visualize?

A) The total budget spent on the project.

B) The number of lines of code written.

C) The remaining work (in hours or points) versus the remaining time in the Sprint.

D) The list of employees who are performing poorly.

> A Burndown chart shows the "ideal" line vs. the "actual" line of work remaining. If the actual line is above the ideal, the team is behind schedule for that Sprint.

Q16. (Scenario-Based) A team is doing **Sprint Planning**. They realize that the top item in the backlog is too large to fit into a single Sprint. What should they do?

A) Make the Sprint 3 months long.

B) Skip that item and move to the next one.

C) Decompose the "Epic" into smaller, manageable "User Stories" that can be finished in one Sprint.

D) Work 20 hours a day to finish it.

> In Agile, work must be "investable" and small enough to be completed within the Sprint timebox. Large features (Epics) must be broken down into smaller pieces.

Q17. (Conceptual) What does the **"A"** stand for in the **INVEST** criteria for good User Stories?

A) Accurate

B) Automated

C) Assignable (or sometimes Actionable, though Assignable is standard)

D) Agile

> Actually, in the most common INVEST model: I-Independent, N-Negotiable, V-Valuable, E-Estimable, S-Small, **T-Testable**. *Correction check*: The "A" in many variants stands for **Assignable**, but the classic acronym is INVEST where A isn't present. However, in some frameworks, "A" refers to **Acceptance Criteria** being clear. (The most professional answer for T is Testable).

Q18. (Scenario-Based) A Scrum team has 12 members. They find that the Daily Stand-up is taking 45 minutes and becoming a status report to the manager. What is the Agile solution?

A) Stop doing the meeting.

B) Make the meeting twice a week.

C) Split the team into two smaller Scrum teams (Pizza-sized teams) to reduce communication overhead.

D) Force everyone to stand on one leg during the meeting.

> Scrum works best with small teams (usually 3–9 members). Larger teams suffer from communication complexity, making the Daily Stand-up ineffective.

Q19. (Conceptual) Which of the following is a primary **Value** stated in the Agile Manifesto?

A) Processes and tools over individuals and interactions.

B) Working software over comprehensive documentation.

C) Contract negotiation over customer collaboration.

D) Following a plan over responding to change.

> The Manifesto values the items on the right (documentation), but values the items on the left (working software) *more*.

Q20. (Scenario-Based) An XP team is using **Continuous Integration**. One developer checks in code that "breaks the build" (the automated tests fail). What should the rest of the team do?

A) Ignore it and keep working on their own features.

B) Stop all other work and help the developer fix the build immediately.

C) Delete the developer’s code.

D) Wait for the weekly "Build Fix" meeting.

> In Agile/XP, a "broken build" is a high-priority emergency. Since the build is the team's single source of truth, it must be fixed immediately so everyone else can continue to integrate their work safely.

Deep Dive: The Science of Scrum Cycles

Scrum is built on **Empiricism** (making decisions based on what is actually observed). This is achieved through three pillars:

1. **Transparency:** All aspects of the process must be visible to those responsible for the outcome (e.g., the Scrum Board).
2. **Inspection:** The team must frequently inspect Scrum artifacts and progress toward a Sprint Goal to detect undesirable variances.
3. **Adaptation:** If an inspector determines that one or more aspects of a process deviate outside acceptable limits, the process or the material being processed must be adjusted.

This is why Agile is often called a "Scientific Method" for software—every Sprint is a 2-4 week experiment to see what works and what doesn't.

Stable Web Resources

- Agile Manifesto (Official)
- Scrum Guide (Official)

Quick Revision Cheat Sheet

- **Manifesto:** Software > Docs; People > Tools; Change > Plan.
- **Scrum:** Roles (PO, SM, Dev), Events (Planning, Stand-up, Review, Retro), Artifacts (Backlogs, Increment).
- **XP:** Pair Programming, TDD, Simplicity.
- **Kanban:** Visualize, Limit WIP, Flow.
- **Velocity:** Points per Sprint. **Story Points:** Relative effort.

END OF SUBTOPIC 03 — MOVING TO SUBTOPIC 04

Subtopic 04: Software Requirements Engineering (15 MCQs)

Quick Notes

- **Requirements Engineering (RE):** The process of establishing the services that the customer requires from a system and the constraints under which it operates.
- **Functional Requirements:** Statements of services the system should provide (e.g., "The system shall allow users to search the product catalog").
- **Non-Functional Requirements (NFRs):** Constraints on the services or functions (e.g., "The system shall respond within 2 seconds"). Often called "Quality Attributes."
- **RE Process:** Elicitation (Gathering) → Analysis (Negotiating) → Specification (Documenting) → Validation (Checking).
- **User Requirements:** Statements in natural language for the customer.

- **System Requirements:** Detailed technical descriptions for the developers.

Common Traps & Confusions

- **Functional vs. Non-Functional:** If a requirement describes "What" the system does, it's Functional. If it describes "How" the system performs (Speed, Security, Reliability), it's Non-Functional.
- **Ambiguity:** Requirements must be verifiable. "The system shall be user-friendly" is a bad requirement because it cannot be measured.
- **Requirements vs. Design:** Requirements define the *Problem* (What); Design defines the *Solution* (How).

MCQs

Q1. (Scenario-Based) A banking client states: "The system must ensure that all financial transactions are encrypted using AES-256 and that the database is backed up every 4 hours." This statement is best classified as:

A) A Functional Requirement

B) A Non-Functional Requirement

C) A User Requirement

D) A Domain Requirement

> This describes "how" the system should behave regarding security and availability, not a specific service or feature provided to the end-user. Therefore, it is a Non-Functional Requirement (NFR).

Q2. (Comparison) What is the primary difference between **Elicitation** and **Analysis** in the RE process?

A) Elicitation is about gathering requirements from stakeholders; Analysis is about resolving conflicts and negotiating priorities among those requirements.

B) Elicitation is for developers; Analysis is for customers.

C) Elicitation happens at the end; Analysis happens at the start.

D) Elicitation is documenting; Analysis is gathering.

> Elicitation (discovery) involves techniques like interviews and surveys. Analysis involves taking that raw information and identifying missing requirements, inconsistencies, or impossibilities.

Q3. (Scenario-Based) During a requirements meeting, a developer says: "The search function must return results in less than 500 milliseconds for a database of 1 million records." This statement is considered **Verifiable** because:

A) It uses a very fast number.

B) It provides a specific, measurable metric that can be tested objectively during the validation phase.

C) It is written by a technical expert.

D) It mentions a database.

> Good requirements must be verifiable (quantifiable). Using time (500ms) and scale (1M records) allows a tester to pass or fail the requirement objectively.

Q4. (Conceptual) Which document serves as the official statement of what the system developers should implement and is often used as part of a legal contract?

A) The Design Document

B) The Project Plan

C) The Software Requirements Specification (SRS)

D) The User Manual

> The SRS is the baseline document for the project. It includes both user and system requirements and serves as the "source of truth" for what the product is supposed to do.

Q5. (Scenario-Based) You are building a flight booking system. A stakeholder mentions: "The system should never allow two people to book the same seat at the same time." This is an example of:

A) A Functional Requirement

B) A Non-Functional Requirement

C) An External Constraint

D) A UI Mockup

> This defines a core service/rule of the application. It describes a "What" (managing seat availability), making it a functional requirement.

Q6. (Code/Console Logic) An engineer is using a tool to track requirements. They create a "Traceability Matrix." What is the primary purpose of the command represented by this logic?

requirement-trace-log

```
REQ-001 -> MODULE-AUTH -> TEST-CASE-LOGIN-05
REQ-002 -> MODULE-PAY -> TEST-CASE-VISA-12
```

A) To write the source code automatically.

B) To ensure every requirement is linked to a specific code module and a test case, ensuring nothing is missed.

C) To calculate the salary of the developers.

D) To find syntax errors in the code.

> Requirements Traceability ensures that every requirement is actually implemented in the code (Forward Traceability) and that every piece of code is there because of a requirement (Backward Traceability).

Q7. (Comparison) Compare **User Requirements** and **System Requirements**. Who is the primary audience for System Requirements?

A) End-users and business managers.

B) Software developers, architects, and testers.

C) Marketing teams and shareholders.

D) The HR department.

> User requirements are for high-level stakeholders (in natural language). System requirements are the technical, detailed specifications used by engineers to build and test the software.

Q8. (Scenario-Based) A project team uses "Ethnography" to gather requirements. This involves:

A) Sending surveys to 1,000 users.

B) Interviewing the CEO of the company.

C) Observing users in their actual work environment to understand their tasks and social interactions.

D) Reading the competitor's user manual.

> Ethnography is an observational technique used to discover requirements that users might not think to mention during an interview because they are so routine or social in nature.

Q9. (Conceptual) What happens during the **Requirements Validation** phase?

A) The team writes the database schema.

B) The customer pays for the software.

C) The requirements are checked for realism, consistency, and completeness to ensure they define the system the customer really wants.

D) The code is compiled and linked.

> Validation is a quality check. It ensures that the SRS document is accurate and that building the system based on it will satisfy the customer.

Q10. (Scenario-Based) A requirement states: "The software should be compatible with both Windows 10 and Linux Ubuntu 22.04." This is a:

A) Functional Requirement

B) Non-Functional Requirement (Portability)

C) Domain Requirement

D) System Design Choice

> Compatibility and Portability are classic non-functional requirements. They place a constraint on the environment in which the functional code must run.

Q11. (Comparison) What is a **Domain Requirement**?

A) A requirement about the website URL.

B) A requirement derived from the specific application area (e.g., medicine, law, physics) that reflects its unique rules and characteristics.

C) A requirement that only applies to the database administrator.

D) A requirement that must be finished in one day.

> Domain requirements come from the industry. For example, a banking app *must* follow specific interest-rate calculation laws. These can be functional or non-functional.

Q12. (Conceptual) Why is **Requirements Management** necessary during the entire lifecycle of a project?

A) To prevent developers from going on vacation.

B) Because requirements change over time as the business environment, stakeholders' needs, and technology evolve.

C) To increase the size of the SRS document.

D) To make the project more expensive.

> Requirement "Volatility" is a fact of life. Requirements management handles the process of change requests and ensures the system evolves without losing integrity.

Q13. (Scenario-Based) An engineer identifies a conflict: The Marketing team wants a high-resolution video player (Functional), but the IT team says the server bandwidth cannot handle it (Non-Functional constraint). This conflict should be resolved during:

A) Elicitation

B) Analysis and Negotiation

C) Validation

D) Maintenance

> Analysis is the phase where trade-offs are discussed and compromises are made between conflicting stakeholder needs.

Q14. (Conceptual) In an Agile environment, how are requirements typically documented instead of a 500-page SRS?

A) They aren't documented at all.

B) Through User Stories and Acceptance Criteria in a Product Backlog.

C) In the source code comments only.

D) On post-it notes that are thrown away daily.

> Agile uses lightweight documentation. User Stories capture the requirement from the user's perspective, and the Product Backlog acts as the living requirements document.

Q15. (Scenario-Based) A stakeholder says: "I want the system to be fast." As a Requirements Engineer, your response should be to:

A) Write it exactly like that in the SRS.

B) Tell them that "fast" is impossible.

C) Ask follow-up questions to define a measurable metric, such as "95% of

transactions should complete in under 1 second."

D) Ignore the comment because it is too vague.

> Your job is to remove ambiguity. A requirement like "fast" is not verifiable. You must translate vague desires into concrete, measurable goals.

Deep Dive: Non-Functional Requirement Conflict

One of the hardest parts of RE is managing the **Inverse Relationship** between certain NFRs. For example:

- **Security vs. Usability:** Increasing security (e.g., requiring 2FA and 20-character passwords) often makes the software harder to use (lower usability).
- **Performance vs. Portability:** Writing code in Assembly for a specific CPU might make it "fast" (high performance) but impossible to run on another system (low portability).

Requirements Engineering is the art of finding the "Optimal Balance" between these competing quality attributes based on the business context.

Stable Web Resources

- Wikipedia: Requirements Engineering
- Modern Analyst: Functional vs. Non-Functional Guide

Quick Revision Cheat Sheet

- **Functional:** "The system shall..." (Service/Action).
- **Non-Functional:** "The system must be..." (Property/Constraint).
- **SRS:** The contract between user and developer.
- **Verifiable:** If you can't measure it, it's not a good requirement.
- **Elicitation:** Interviews, Surveys, Workshops, Observation.
- **Validation:** Are we building the *right* thing?

END OF SUBTOPIC 04 — MOVING TO SUBTOPIC 05

Subtopic 05: Software Project Management (15 MCQs)

Quick Notes

- **Project Management:** The planning, monitoring, and control of the people, process, and events that occur as software evolves from a preliminary concept to operational deployment.
- **The 4 P's:** People (the most important), Product (the software to be built), Process (the framework), and Project (the actual work).
- **Software Estimation:** Determining how much money, effort, resources, and time a project will take. Models include **COCOMO** (Cost Constructive Model) and **Function Points**.
- **Critical Path Method (CPM):** The longest sequence of tasks in a project plan which must be completed on time for the project to complete on time.
- **Earned Value Analysis (EVA):** A technique for measuring project progress by comparing planned work versus actual work performed.

Common Traps & Confusions

- **Effort vs. Duration:** Effort is measured in Person-Months (total work). Duration is the calendar time. Adding people reduces duration (up to a point) but usually *increases* total effort due to communication overhead.
- **Critical Path vs. Slack:** Tasks on the critical path have **Zero Slack** (or float). If they are delayed, the whole project is delayed.
- **LOC vs. Function Points:** Lines of Code (LOC) is technology-dependent (C++ needs more lines than Python for the same task). Function Points (FP) are technology-independent (focusing on functionality).

MCQs

Q1. (Scenario-Based) A software project is currently 2 weeks behind schedule. The manager decides to move two senior developers from another project to this one to speed things up. According to Brooks's Law, what is the most likely outcome?

A) The project will finish exactly on time.

B) The project will likely fall even further behind schedule due to the communication overhead of bringing the new developers up to speed.

C) The project will finish early because senior developers work faster.

D) The quality of the code will improve, but the timeline will stay the same.

> Brooks's Law states that "adding manpower to a late software project makes it later." The time required for new members to learn the system and for existing members to teach them outweighs the extra work capacity they provide in the short term.

Q2. (Calculation) You are using the **Basic COCOMO** model where Effort (E) = 2.4 * (KLOC)^1.05. If your project is estimated at 10,000 lines of code (10 KLOC), what is the estimated effort in Person-Months (rounded)?

A) 10 PM

B) 27 PM

C) 2.4 PM

D) 105 PM

> Calculation: E = 2.4 * (10)^1.05. Since 10^1.05 is slightly more than 10 (approx 11.22), then 2.4 * 11.22 = 26.928. Rounding gives 27 Person-Months.

Q3. (Comparison) When comparing **Gantt Charts** and **PERT/CPM Charts**, which statement is true regarding their primary usage?

A) Gantt charts show task dependencies; PERT charts show time.

B) Gantt charts are best for visualizing the project schedule over time; PERT/CPM charts are best for identifying the Critical Path and task interdependencies.

C) Gantt charts are only for Agile; PERT is only for Waterfall.

D) PERT charts are used for budget tracking; Gantt is for code quality.

> Gantt charts provide a timeline-based view of when tasks start and end. PERT (Program Evaluation and Review Technique) uses a network diagram to show how tasks depend on one another and to calculate which tasks are "critical."

Q4. (Code/Console Logic) A project manager is reviewing a task's metadata in a project management tool (like Jira or Azure DevOps). Based on the JSON representation below, what is the **Slack Time** for this task?

task-metadata.json

```
{
"task_id": "FE-04",
"early_start": 10,
"late_start": 14,
"duration_days": 5
}
```

A) 10 days

B) 5 days

C) 4 days

D) 0 days

> Slack (or Float) is calculated as (Late Start - Early Start) or (Late Finish - Early Finish). Here, 14 - 10 = 4 days. This task can be delayed for 4 days without affecting the overall project deadline.

Q5. (Scenario-Based) A team of 5 developers has 10 communication channels. If the manager adds 2 more developers to the team, how many **total communication channels** will there be now?

A) 12

B) 14

C) 21

D) 25

> The formula for communication channels is n(n - 1) / 2. For 7 developers: 7(6) / 2 = 42 / 2 = 21. This exponential growth explains why adding people to a project increases complexity.

Q6. (Conceptual) What is the primary advantage of using **Function Points (FP)** over **Source Lines of Code (SLOC)** for size estimation?

A) SLOC is easier to calculate before the code is written.

B) FP is independent of the programming language and focuses on the functionality delivered to the user.

C) FP only measures the complexity of the database.

D) SLOC is better for estimating UI/UX effort.

> Function Points measure the "logical" size of the system (inputs, outputs, queries, files) regardless of whether it is built in Java, Python, or C++. SLOC is highly variable and can only be accurately known after the code is written.

Q7. (Scenario-Based) A project manager notices that the **CPI (Cost Performance Index)** of a project is 0.85. What does this indicate about the project's health?

A) The project is ahead of schedule.

B) The project is under budget.

C) The project is over budget (spending more than planned for the work achieved).

D) The project has too many bugs.

> CPI = Earned Value (EV) / Actual Cost (AC). A CPI < 1.0 means you are getting less than $1 worth of work for every $1 spent. 0.85 indicates the project is significantly over budget.

Q8. (Code/Console Logic) A developer checks the project status via a command-line tool. Based on the output, which task is currently the **Bottleneck**?

pm-tool --status

```
Task ID | Status | Dependency | Slack
-----------------------------------------
T1 | Completed | None | 0
T2 | In Progress | T1 | 0
T3 | Waiting | T2 | 0
T4 | In Progress | T1 | 8
```

A) T1

B) T2

C) T3

D) T4

> T2 is on the **Critical Path** (Slack = 0) and is currently "In Progress." Since T3 is waiting for T2 and there is zero slack, any delay in T2 will delay the entire project. T4 has 8 days of slack, so it is not a bottleneck.

Q9. (Conceptual) In software project management, what is the difference between a **Milestone** and a **Deliverable**?

A) Milestones are for developers; Deliverables are for managers.

B) A Milestone is a point in time (an event) used to measure progress; a Deliverable is a tangible work product (like code or a manual) given to the customer.

C) Milestones are paid for; Deliverables are free.

D) There is no difference.

> Milestones mark the completion of a major phase (e.g., "Design Approved"). They have zero duration. Deliverables are the actual results produced (e.g., "The SRS Document").

Q10. (Scenario-Based) A client keeps requesting "just one more small button" or "just a slight change in color" every week without extending the deadline or budget. This phenomenon is known as:

A) Agile Development

B) Feature Creep

C) Scope Creep

D) Gold Plating

> Scope Creep refers to uncontrolled changes or continuous growth in a project's scope without corresponding adjustments to time, cost, and resources. (Gold Plating is when the *developer* adds features the client didn't ask for).

Q11. (Calculation) A software project has 5 external inputs, 2 external outputs, and 3 internal logical files. All are of "Average" complexity. If the weighing factor for Input is 4, Output is 5, and Logical File is 10, what is the **Unadjusted Function Point (UFP)** count?

A) 10

B) 40

C) 60

D) 100

> UFP = (5 * 4) + (2 * 5) + (3 * 10) = 20 + 10 + 30 = 60.

Q12. (Comparison) What is the difference between **Reactive** and **Proactive** risk management?

A) Reactive management handles risks only when they become actual problems; Proactive management identifies risks early and develops contingency plans.

B) Reactive is for Waterfall; Proactive is for Agile.

C) Proactive management is only for security risks.

D) Reactive management is always better for small teams.

> Proactive management is the hallmark of a mature engineering process. It involves "Risk Assessment" and "Risk Control" before the project begins to minimize the impact of potential failures.

Q13. (Conceptual) The **W5HH Principle** by Barry Boehm helps project managers define the project. What do the two 'H's stand for?

A) Hardware and Human Resources.

B) High-level and Help-desk.

C) How many (people) and How much (cost).

D) History and Header.

> W5HH stands for: Why, What, When, Who, Where, How much, and How many. It is a checklist for project planning.

Q14. (Scenario-Based) A manager is forming a team for a project with very high innovation but a very short timeline. Which team structure is most suitable?

A) Controlled Centralized (CC) - Top-down authority.

B) Democratic Decentralized (DD) - Collaborative and consensus-based.

C) Controlled Decentralized (CD) - Sub-groups with a leader.

D) Hierarchical - Rigid military-style.

> For high-innovation projects where creative problem-solving is needed, a Democratic Decentralized structure allows for better idea exchange. CC is better for simple, repetitive tasks.

Q15. (Code/Console Logic) A lead engineer is setting up a configuration for a project monitoring script. Which metric is being calculated by this logic?

monitoring-script.py

```python
def check_health(budget_spent, work_completed_value):
# work_completed_value is the "Earned Value"
return work_completed_value / budget_spent
```

A) Schedule Variance (SV)

B) Cost Performance Index (CPI)

C) Lines of Code per Day

D) Defect Density

> The ratio of Earned Value (value of work actually performed) to Actual Cost (budget spent) is the definition of CPI. A result > 1.0 is excellent.

Deep Dive: Software Effort Estimation Mechanics

Estimation is not a "guess"; it is a statistical process. The **Intermediate COCOMO** model improves upon the basic one by introducing 15 **Cost Drivers** (called Effort Multipliers). These include:

- **Product Attributes:** Required reliability, database size, product complexity.
- **Hardware Attributes:** Execution time constraints, memory constraints.
- **Personnel Attributes:** Analyst capability, developer experience with the language.
- **Project Attributes:** Use of software tools, required development schedule.

If a team is highly experienced (Personnel Attribute), the multiplier might be 0.70 (reducing effort). If the project has an ultra-tight schedule (Project Attribute), the multiplier might be 1.23 (increasing effort).

Stable Web Resources

- Wikipedia: COCOMO Model

Quick Revision Cheat Sheet

- **People, Product, Process, Project:** The 4 P's.
- **Critical Path:** Longest path, zero slack. Delay here = Project delay.
- **CPM/PERT:** Dependency focus. **Gantt:** Timeline focus.
- **CPI = EV / AC:** Health check (Value / Cost).
- **Brooks's Law:** New people = More communication = Slower project.
- **COCOMO:** Estimation based on KLOC and Cost Drivers.

END OF SUBTOPIC 05 — MOVING TO SUBTOPIC 06

Subtopic 06: Software Design (15 MCQs)

Quick Notes

- **Software Design:** The process of defining software methods, functions, objects, and the overall structure of the code to satisfy requirements.
- **Abstraction:** Hiding the internal implementation details and showing only the essential features of the object.
- **Modularity:** Dividing a system into separate, independent components (modules) that can be developed and tested individually.
- **Cohesion:** A measure of how strongly related the functions within a single module are. **Goal: High Cohesion.**
- **Coupling:** A measure of how much one module relies on or is "connected" to another module. **Goal: Low/Loose Coupling.**
- **Information Hiding:** Restricting access to a component's internal data (often using 'private' keywords).

Common Traps & Confusions

- **Cohesion vs. Coupling:** Cohesion is *internal* (inside the module); Coupling is *external* (between modules). Think: "Cohesion is how well the glue holds the module together; Coupling is how many strings are tied between different modules."
- **Design vs. Architecture:** Architecture is the "Big Picture" (high-level components); Design is the "Blueprints" (how the specific classes and methods work).
- **Over-Engineering:** Creating complex abstractions for simple problems, making the code harder to read.

MCQs

Q1. (Scenario-Based) A developer is writing a "PaymentModule." They include functions for calculating taxes, generating PDF invoices, sending emails to customers, and updating the database. A senior reviewer says the module has **Low Cohesion**. Why?

A) Because the module is too small.

B) Because the module is performing too many unrelated tasks that should be handled by separate specialized modules.

C) Because the module doesn't use enough global variables.

D) Because the module is written in C++ instead of Python.

> High cohesion means a module should do one thing and do it well. Mixing tax calculation, PDF generation, and email logic makes the module difficult to maintain and re-use.

Q2. (Code/Console Logic) Analyze the following Python code snippet. Which design principle is being violated here?

bad_design.py

```python
class Engine:
    def __init__(self):
        self.temperature = 90

class Car:
    def check_engine(self, engine_obj):
        # Directly modifying internal data of another class
        if engine_obj.temperature > 100:
```

```
            engine_obj.temperature = 90
```

A) Inheritance

B) Polymorphism

C) Information Hiding (Encapsulation)

D) Functional Programming

> The `Car` class is directly accessing and modifying the `temperature` attribute of the `Engine` class. This creates tight coupling and violates Information Hiding. The engine should provide a method like `cool_down()` instead.

Q3. (Conceptual) In software design, what is the primary goal of **Refactoring**?

A) To add new features to the software.

B) To improve the internal structure of the code without changing its external behavior.

C) To convert a desktop application into a web application.

D) To find and delete all comments in the code.

> Refactoring is "cleaning up" the design of existing code (reducing coupling, increasing cohesion) to make it more maintainable and readable, without altering what the code actually does.

Q4. (Comparison) What is the difference between **Conceptual Design** and **Technical Design**?

A) Conceptual design explains "what" the system does for the user; Technical design explains "how" the system is implemented for the developer.

B) Conceptual design uses code; Technical design uses diagrams.

C) Technical design is done by the customer; Conceptual by the manager.

D) There is no difference.

> Conceptual design (high-level) is often represented by Data Flow Diagrams or UI mockups. Technical design (low-level) involves class diagrams, database schemas, and API definitions.

Q5. (Scenario-Based) You are designing a weather application. You want to ensure that if the "Data Source" changes from a local database to an online API, the "UI Module" does not need to be changed at all. This is achieved by:

A) Increasing Coupling.

B) Establishing an Interface/Abstraction between the UI and the Data Source.

C) Removing all functions from the code.

D) Using only global variables.

> By using an interface, the UI depends on a "contract" rather than a specific implementation. This "Loose Coupling" allows you to swap out the backend logic without breaking the frontend.

Q6. (Code/Console Logic) A developer is looking at a system log that tracks object interactions. Which type of **Coupling** is most likely present if Module A can modify the local data of Module B?

coupling-report.txt

```
[WARN] Module_Auth modified variable 'is_admin'
       inside Module_UI memory space.
```

A) Data Coupling

B) Stamp Coupling

C) Content Coupling (High/Bad)

D) External Coupling

> Content coupling occurs when one module modifies or relies on the internal workings/data of another. This is the highest (worst) form of coupling.

Q7. (Conceptual) Which of the following is the best example of **Functional Cohesion**?

A) A module that performs all tasks related to the "Monday Morning" schedule.

B) A module that performs exactly one mathematical calculation, such as "Compute Square Root."

C) A module that contains all the variables used in the entire program.

D) A module that groups functions because they all use the same printer.

> Functional cohesion is the strongest type of cohesion, where every part of the module contributes to a single, well-defined task.

Q8. (Scenario-Based) A design uses **Inheritance** where a "Circle" class inherits from a "Shape" class. This is an example of which design relationship?

A) "HAS-A" relationship

B) "IS-A" relationship

C) "PART-OF" relationship

D) "USES-A" relationship

> Inheritance represents an "IS-A" relationship (A Circle is a Shape). Composition/Aggregation represents a "HAS-A" relationship (A Car has an Engine).

Q9. (Comparison) What is the relationship between **Redundancy** and **Reliability** in software design?

A) Redundancy makes software less reliable.

B) Strategic redundancy (e.g., backup systems) can improve reliability by providing a failover if one component fails.

C) Redundancy is the same thing as high coupling.

D) Redundancy is only used in database design, not software design.

> While redundant code is bad for maintenance, redundant components (like multiple servers or data backups) are essential for high-reliability systems.

Q10. (Conceptual) What does the **Open-Closed Principle** (from SOLID) suggest?

A) Software should be open to everyone to read, but closed for editing.

B) Software entities (classes, modules) should be open for extension but closed for modification.

C) All code should be open source.

D) Databases should be closed when the application is running.

> This means you should be able to add new functionality (extension) by adding new code, rather than by changing old code that is already tested and working.

Q11. (Code/Console Logic) An engineer is reviewing a **UML Class Diagram** represented in a textual format. What relationship exists between `Library` and `Book`?

uml-spec.txt

```
Library <>--* Book
(Library contains many Books. If Library is destroyed,
```

```
 Books can still exist independently.)
```

A) Composition

B) Aggregation

C) Generalization

D) Realization

> In UML, the hollow diamond <>- - represents Aggregation. It is a "part-of" relationship where the child (Book) can exist independently of the parent (Library). A solid diamond would represent Composition (stronger bond).

Q12. (Scenario-Based) A design choice is made to use a **Design Pattern** to solve a recurring problem of creating objects without specifying their exact class. This is known as:

A) The Singleton Pattern

B) The Observer Pattern

C) The Factory Pattern

D) The Strategy Pattern

> The Factory Method pattern provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created.

Q13. (Comparison) Compare **Top-Down Design** and **Bottom-Up Design**. Which is more common in modern Object-Oriented programming?

A) Top-Down, because you start with the main function.

B) Bottom-Up, because you design and test low-level classes (like Data models) first and then combine them into larger systems.

C) Top-Down, because it requires no testing.

D) Both are used equally in every single project.

> Object-Oriented design often favors a bottom-up approach where foundational objects are identified and built first, eventually forming the complete system logic.

Q14. (Conceptual) What is the primary purpose of a **Data Flow Diagram (DFD)** in the design phase?

A) To show the memory addresses of variables.

B) To model how information moves through a system and the transformations applied to that data.

C) To design the user interface buttons.

D) To list all the hardware requirements.

> DFDs focus on the movement of data between processes, data stores, and external entities. They are a logical model of the system's "data metabolism."

Q15. (Scenario-Based) A developer is worried about "Single Point of Failure" in their system design. They decide to use a **Load Balancer** to distribute traffic across three identical servers. This is a design decision related to:

A) Usability

B) Availability and Fault Tolerance

C) Portability

D) Testability

> Availability is the percentage of time a system is operational. By removing a single point of failure (one server) and adding redundancy, the designer improves the system's ability to stay online during a crash.

Deep Dive: Coupling Levels (The Myers Scale)

In 1974, Glenford Myers defined a scale of coupling from worst to best. Professional designers aim for the bottom of this list:

1. **Content Coupling (Worst):** One module reaches into the private memory/code of another.
2. **Common Coupling:** Multiple modules share the same global data area. (Danger: If data format changes, all modules break).
3. **Control Coupling:** One module tells another what to do by passing a "flag" (e.g., `if flag==1 do A else do B`).
4. **Stamp Coupling:** Passing a whole complex data structure (like a `UserObject`) when only one small piece of it (like `UserID`) is needed.
5. **Data Coupling (Best):** Modules only communicate by passing simple data arguments (integers, strings).

Stable Web Resources

- Refactoring Guru: Design Patterns
- Wikipedia: Software Design

Quick Revision Cheat Sheet

- **High Cohesion:** The module does one task well (Good).
- **Low Coupling:** Modules are independent and changes don't ripple (Good).
- **Abstraction:** Hide the "How", show the "What".
- **Encapsulation:** Data and methods live together; keep variables private.
- **SOLID:** Five principles for better OO design.
- **Refactoring:** Improve internal design without changing external behavior.

END OF SUBTOPIC 06 — MOVING TO SUBTOPIC 07

Subtopic 07: Software Architecture (15 MCQs)

Quick Notes

- **Software Architecture:** The high-level structure of a software system, the discipline of creating such structures, and the documentation of these structures. It deals with "The Big Picture."
- **Architecture Patterns:**
  - *Layered:* Groups functionality into layers (e.g., Presentation, Logic, Data).
  - *Client-Server:* Distributed application structure between providers (servers) and requesters (clients).
  - *Microservices:* System composed of small, independent services communicating over a network.

- *Model-View-Controller (MVC):* Separates data (Model), UI (View), and logic (Controller).
- *Pipes and Filters:* Processes a stream of data through a sequence of processing components.

- **Architectural Views:** Often described using the **4+1 View Model** (Logical, Process, Development, Physical + Use Case view).
- **Non-Functional Requirements (NFRs):** Architecture is primarily driven by NFRs like Scalability, Availability, and Security.

Common Traps & Confusions

- **Architecture vs. Design:** Architecture involves decisions that are "expensive to change" later (e.g., choosing a database type). Design involves local decisions (e.g., choosing a sorting algorithm inside a class).
- **Microservices Complexity:** While microservices allow independent scaling, they introduce massive network complexity and data consistency challenges compared to a Monolith.
- **Layered Architecture "Leaking":** A common mistake where the UI layer talks directly to the database, bypassing the business logic layer. This is called "Layer Leakage."

MCQs

Q1. (Comparison) A senior architect states: "Design is what we do when we want to satisfy local requirements; Architecture is what we do when we want to satisfy global system properties." What is a classic example of an **Architectural** decision?

A) Choosing between a `for` loop and a `while` loop.

B) Deciding to use `CamelCase` for variable names.

C) Deciding whether to use a Centralized Database or a Distributed Microservices data store.

D) Implementing a specific bubble sort algorithm.

> Architecture deals with high-level structure and system-wide properties. Choosing the data storage strategy is a foundational decision that affects the entire system and is very difficult to change later.

Q2. (Scenario-Based) You are designing a system for a news website. The owner wants to ensure that the user interface can be completely redesigned every month without affecting the underlying database or the news-writing logic. Which pattern is best?

A) Pipes and Filters

B) Repository Pattern

C) Model-View-Controller (MVC)

D) Master-Slave

> MVC is designed to decouple the UI (View) from the data (Model) and the logic (Controller). This allows the View to be changed or swapped out independently of the rest of the system.

Q3. (Code/Console Logic) An engineer is looking at a network trace of a system. Based on the port usage and communication style, which **Architectural Pattern** is most likely in use?

network-sniffer.log

```
[REQ] 192.168.1.5:54322 -> 10.0.0.5:80 (GET /index.html)
[RES] 10.0.0.5:80 -> 192.168.1.5:54322 (200 OK)
[REQ] 192.168.1.12:51201 -> 10.0.0.5:80 (GET /style.css)
```

A) Client-Server

B) Peer-to-Peer

C) Pipes and Filters

D) Blackboard

> The log shows multiple unique IP addresses (Clients) making requests to a single central IP address (Server) on a standard web port (80). This is the characteristic "many-to-one" relationship of a Client-Server architecture.

Q4. (Conceptual) In the **4+1 View Model**, which view is primarily concerned with the system's performance, scalability, and throughput by describing how the system is executed on physical hardware?

A) Logical View

B) Process View

C) Development View

D) Physical View

> The Process View describes the communication between processes and handles non-functional requirements like concurrency and performance. (The Physical View handles the mapping of software to hardware).

Q5. (Scenario-Based) A large e-commerce platform finds that during peak sales, the "Payment Service" crashes due to high load, taking down the entire website. The architect suggests moving to **Microservices**. What is the primary architectural benefit here?

A) It makes the code easier to write.

B) Fault Isolation: If the Payment service fails, other services like "Product Search" can remain operational.

C) It reduces the total number of servers needed.

D) It eliminates the need for a database.

> In a monolith, one failure can crash the whole process. In microservices, services are isolated. This increases the system's overall "Availability" through compartmentalization.

Q6. (Code/Console Logic) A developer is looking at an **Event-Driven** system log. Which component in this architecture acts as the "Broker"?

event-bus.log

```
[PUB] Order_Service: Published "OrderCreated"
[BUS] Routing "OrderCreated" to [Email_Service, Inventory_Service]
[SUB] Email_Service: Received "OrderCreated"
```

A) Order_Service

B) The [BUS] (Event Bus / Broker)

C) Email_Service

D) The Developer

> In an Event-Driven (Pub-Sub) architecture, the Broker (or Bus) is responsible for receiving events from publishers and routing them to the correct subscribers, decoupling the services from each other.

Q7. (Comparison) What is the difference between **Horizontal Scaling** and **Vertical Scaling**?

A) Horizontal is for RAM; Vertical is for CPU.

B) Horizontal scaling adds more machines to the pool; Vertical scaling adds more power (CPU/RAM) to an existing machine.

C) Vertical scaling is only for cloud environments.

D) Horizontal scaling is always cheaper.

> Vertical scaling (Scale-Up) has a hard physical limit. Horizontal scaling (Scale-Out) allows for near-infinite growth by adding more low-cost server nodes to a cluster.

Q8. (Scenario-Based) You are designing a data-processing pipeline for a bank where transactions must pass through: "Security Check" -> "Currency Conversion" -> "Logging". Each step must be independent and reusable. Which pattern is best?

A) Layered Architecture

B) MVC

C) Pipes and Filters

D) Blackboard

> Pipes and Filters are ideal for systems that perform a series of discrete transformations on data. Each "Filter" is an independent unit, and they are connected via "Pipes" (data streams).

Q9. (Conceptual) In a **Layered Architecture**, what does the "Strict Layering" rule mandate?

A) Every layer must be written in a different language.

B) A layer can only access the services of the layer immediately below it.

C) All layers must use the same database.

D) There can be no more than three layers.

> Strict layering prevents tight coupling across the system. If Layer 4 can only talk to Layer 3, you can replace Layer 2 without Layer 4 even knowing it changed.

Q10. (Scenario-Based) A company is using a **Shared Repository** (Blackboard) architecture. Several different tools (AI analyzer, Manual editor, Statistics generator) all work on the same central database. What is the biggest risk of this architecture?

A) The tools will be too small.

B) The central repository becomes a bottleneck and a single point of failure.

C) It is impossible to use SQL.

D) Tools cannot read data from each other.

> While Shared Repository patterns are great for data integration, the central store must handle all concurrency and load. If it goes down, every tool in the system stops working.

Q11. (Code/Console Logic) Analyze the structure of the following URI. What architectural style does this represent?

```
api-request

GET /api/v1/customers/502/orders
Content-Type: application/json
```

A) SOAP

B) REST (Representational State Transfer)

C) GraphQL

D) RPC (Remote Procedure Call)

> REST architecture uses standard HTTP methods (GET) and structured URIs that represent resources (customers, orders) rather than specific function calls.

Q12. (Comparison) Compare **Monolithic** and **Service-Oriented Architecture (SOA)**. What is a key characteristic of SOA?

A) SOA uses only one programming language.

B) SOA provides services to other components via a communications protocol over a network (interoperability).

C) SOA is faster because it doesn't use a network.

D) Monolithic systems are better for large teams.

> SOA (and its successor, Microservices) focuses on "Services" as discrete units of functionality that can be reused across different applications via standard interfaces.

Q13. (Conceptual) What is the purpose of an **API Gateway** in a Microservices architecture?

A) To store the database passwords.

B) To act as a single entry point that routes requests to the appropriate internal microservices.

C) To compile the code of all services into one file.

D) To replace the user's browser.

> The API Gateway simplifies the client's job. Instead of the client needing to know the address of 50 different microservices, it talks to the Gateway, which handles routing, authentication, and load balancing.

Q14. (Scenario-Based) An architect is using the **ATAM (Architecture Trade-off Analysis Method)**. The goal is to:

A) Automatically generate code from diagrams.

B) Identify "Trade-off points" where a decision that improves one quality (like Security) might hurt another (like Performance).

C) Calculate the total cost of developers' salaries.

D) Choose the cheapest laptop for the team.

> ATAM is a systematic way to evaluate architecture. It recognizes that you can't have "everything." For example, adding encryption (Security) adds processing time (Latency/Performance). Architects must find the right balance.

Q15. (Conceptual) In the **Model-View-Controller** pattern, which component is responsible for updating the state of the database?

A) Model

B) View

C) Controller

D) Event Bus

> The Model represents the data and the business rules. It is the only component that interacts with the database or data source directly.

Deep Dive: The Fallacies of Distributed Computing

When moving from Monolithic Architecture to Distributed Architecture (Microservices/SOA), architects must avoid the 8 "Fallacies" identified by L. Peter Deutsch:

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology doesn't change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

Failing to account for these (e.g., assuming a network call will always succeed) leads to "Fragile Architecture" where the system crashes as soon as the network jitters.

Stable Web Resources

- Wikipedia: Software Architecture
- Martin Fowler: Software Architecture Guide

Quick Revision Cheat Sheet

- **Architecture:** Global, expensive to change. **Design:** Local, internal logic.
- **Layered:** Separation of concerns (UI, Logic, DB).
- **Microservices:** Scalable, independent, complex networking.
- **MVC:** Model (Data), View (UI), Controller (Logic).
- **4+1 Views:** Logical, Process, Development, Physical, Scenarios.
- **Scalability:** Horizontal (More nodes) vs Vertical (Better node).

END OF SUBTOPIC 07 — MOVING TO SUBTOPIC 08

Subtopic 08: User Interface Design (10 MCQs)

Quick Notes

- **User-Centered Design (UCD):** A design philosophy where the needs, wants, and limitations of end-users are given extensive attention at each stage of the design process.
- **Golden Rules (Shneiderman):** Strive for consistency, enable frequent users to use shortcuts, offer informative feedback, design dialogs to yield closure, and support internal locus of control.
- **Usability:** The degree to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction.
- **Accessibility:** Ensuring the UI is usable by people with disabilities (e.g., screen reader support, high contrast).
- **Mental Models:** The user's internal understanding of how the system works. Good UI matches the user's mental model, not the system's technical implementation.

Common Traps & Confusions

- **UI vs. UX:** UI (User Interface) is the specific set of screens, buttons, and icons. UX (User Experience) is the overall "feeling" and logic of the interaction.
- **Recognition vs. Recall:** UI should minimize the user's memory load. It is easier to *recognize* a command from a menu than to *recall* a specific command name to type.
- **Aesthetic-Usability Effect:** Users often perceive more aesthetically pleasing designs as more usable, even if they are not.

MCQs

Q1. (Scenario-Based) A user is filling out a long registration form. After clicking "Submit," the page clears all fields and shows a red error message at the top saying "Invalid Input,"

but doesn't highlight which field is wrong. This is a violation of which of Shneiderman's Golden Rules?

A) Strive for consistency.

B) Enable frequent users to use shortcuts.

C) Offer informative feedback and simple error handling.

D) Design dialogs to yield closure.

> Informative feedback requires that the system tells the user exactly what went wrong and how to fix it. Simply saying "Invalid Input" without indicating the field is poor UI design.

Q2. (Comparison) What is the primary architectural difference between **Direct Manipulation** (like a GUI) and **Command Line Interfaces (CLI)**?

A) CLI is always faster for everyone.

B) Direct manipulation relies on recognition (seeing icons/menus), while CLI relies on recall (remembering syntax).

C) Direct manipulation does not require a computer monitor.

D) CLI is more modern than GUI.

> GUIs are generally easier for novices because they only need to recognize symbols. CLIs require the user to have memorized specific commands and parameters, which increases cognitive load.

Q3. (Conceptual) In the context of **Accessibility**, what is the primary purpose of "ALT" tags in HTML or ARIA labels in web applications?

A) To make the website load faster on mobile devices.

B) To hide secret information from the user.

C) To provide a text description of visual elements for users utilizing screen readers.

D) To change the color of the text.

> Accessibility ensures that users with visual impairments can "hear" what is on the screen. Screen readers read the ALT text aloud to explain the purpose of images or buttons.

Q4. (Scenario-Based) A mobile app designer notices that users frequently accidentally click the "Delete" button because it is placed right next to the "Save" button and is very small. Which UI law should the designer apply to fix the button size and placement?

A) Moore's Law

B) Brooks's Law

C) Fitts's Law

D) Pareto's Law

> Fitts's Law states that the time to acquire a target (like a button) is a function of the distance to and the size of the target. Making the button larger and moving it away from conflicting actions reduces errors.

Q5. (Code/Console Logic) A developer is checking a **Web Accessibility** report. Based on the following JSON configuration for a button, what is the most likely UI issue?

audit_report.json

```
{
  "element": "button",
  "bg_color": "#FFFFFF",
  "text_color": "#F1F1F1",
  "label": "Submit"
}
```

A) The label is too long.

B) Low color contrast between the background and text makes it difficult to read for many users.

C) The button has no function.

D) JSON does not support buttons.

> The background color (#FFFFFF is white) and text color (#F1F1F1 is very light gray) are almost identical. This violates accessibility standards for minimum contrast ratios.

Q6. (Conceptual) What does the term **"Affordance"** mean in UI design?

A) The cost of building the interface.

B) The visual properties of an object that suggest how it should be used (e.g., a button "looks" clickable).

C) The number of pixels in an icon.

D) The speed of the server response.

> Affordances are clues about how an object works. A 3D-shaded button "affords" pushing, while a underlined blue text "affords" clicking as a link.

Q7. (Scenario-Based) An expert user of a photo editor wants to quickly switch between the "Brush" and "Eraser" tools. Instead of clicking the icons in the sidebar, they press 'B' and 'E' on the keyboard. This UI feature is an example of:

A) An Affordance.

B) An Accelerator (Shortcut).

C) A Modal Dialog.

D) Informative Feedback.

> Accelerators allow frequent users to perform tasks faster. They "accelerate" the interaction for power users without confusing novice users.

Q8. (Comparison) Compare **Modal** and **Modeless** dialog boxes. Which is the defining characteristic of a Modal dialog?

A) It interrupts the user workflow and requires the user to interact with it before they can return to the main application.

B) It can be minimized while the user works on other things.

C) It is always green.

D) It is only used for showing images.

> Modal dialogs (like "Are you sure you want to quit?") block the rest of the application until they are dismissed. Modeless dialogs (like a "Find/Replace" box) allow the user to continue working elsewhere.

Q9. (Conceptual) In the UI design process, what is the primary purpose of creating a **Low-Fidelity Prototype** (like a paper sketch)?

A) To test the final execution speed of the code.

B) To quickly iterate on layouts and workflows with stakeholders without spending time on code or high-end graphics.

C) To serve as the final documentation for the user.

D) To replace the requirement engineering phase.

> Low-fidelity prototypes are cheap and fast. They allow designers to fail early and fix logical flow issues before any expensive implementation begins.

Q10. (Scenario-Based) A designer is choosing icons for a global application. They decide not to use a "Mailbox" icon for 'Email' because users in different countries have very different looking physical mailboxes. This is a consideration of:

A) Technical constraints.

B) Cultural Diversity and Localization.

C) Screen resolution.

D) Software cohesion.

> Icons should be culturally neutral or localized. Symbols that are common in one culture (like a US-style mailbox) may be completely unrecognizable in another.

Deep Dive: The Action Cycle (Norman's Model)

Don Norman, the author of "The Design of Everyday Things," described the Seven Stages of Action that a user goes through when interacting with an interface:

1. **Forming the Goal:** What does the user want to do?
2. **Forming the Intention:** Deciding to use the software to reach the goal.
3. **Specifying the Action:** Which button do I press?
4. **Executing the Action:** Physically clicking the button.
5. **Perceiving the State:** Seeing the screen change.
6. **Interpreting the State:** Understanding what the change means.
7. **Evaluating the Outcome:** Did I reach my goal?

UI design failures often occur at the **Gulf of Execution** (User doesn't know how to do it) or the **Gulf of Evaluation** (User doesn't know if they succeeded).

Stable Web Resources

- Nielsen Norman Group: 10 Usability Heuristics
- Wikipedia: User Interface Design

Quick Revision Cheat Sheet

- **UCD:** Design for the user, not the developer.
- **Consistency:** Buttons/Colors should mean the same thing across the app.
- **Fitts's Law:** Big targets close to the cursor are easiest to hit.
- **Affordance:** The visual hint of how a button works.
- **Accessibility:** High contrast, ALT text, screen reader friendly.
- **Heuristic Evaluation:** Expert review of UI based on "rules of thumb."

END OF SUBTOPIC 08 — MOVING TO SUBTOPIC 09

Subtopic 09: Software Implementation & Coding (10 MCQs)

Quick Notes

- **Implementation:** The process of translating a system design into an executable system. It includes coding, debugging, and unit testing.
- **Coding Standards:** A set of guidelines (naming conventions, indentation, file structure) that ensure code readability and maintainability across a team.
- **Defensive Programming:** A form of design intended to ensure the continuing function of a piece of software under unforeseen circumstances (e.g., validating all inputs).
- **Code Refactoring:** Restructuring existing computer code—changing the factoring—without changing its external behavior.
- **Technical Debt:** The implied cost of additional rework caused by choosing an easy (fast) solution now instead of using a better approach that would take longer.

Common Traps & Confusions

- **Coding vs. Programming:** Coding is the act of writing syntax. Programming involves the full lifecycle including algorithm design, logic, and problem-solving.
- **Comments:** "Good code documents itself." Over-commenting obvious things (e.g., `i++ // increment i`) is a sign of poor coding style. Comments should explain the *Why*, not the *What*.
- **"It works on my machine":** This is the classic implementation trap. Professional implementation requires portable code and environment-independent configurations.

MCQs

Q1. (Scenario-Based) A developer is writing a function to process user ages. Even though the documentation says users will only provide numbers, the developer adds code to check if the input is negative or a string, and returns an error message if so. This practice is known as:

A) Extreme Programming

B) Premature Optimization

C) Defensive Programming

D) Hardcoding

> Defensive programming involves writing code that anticipates potential errors and handles them gracefully, even if the "official" requirements assume inputs will always be correct.

Q2. (Code/Console Logic) A developer has finished a feature and is preparing to share it with the team. Which **Version Control** command sequence correctly represents the implementation workflow?

```
git-terminal

$ git add .
$ git commit -m "Implement user authentication logic"
$ git push origin feature-auth
```

A) It deletes the local code.

B) It stages the changes, saves a local snapshot with a descriptive message, and uploads it to the shared repository.

C) It compiles the code and runs unit tests.

D) It creates a new branch and switches to it.

> This is the standard implementation workflow: `add` (stage), `commit` (save locally), and `push` (share with the team).

Q3. (Conceptual) Why do professional software organizations enforce **Coding Standards**?

A) To make the code run faster on the CPU.

B) To reduce the "Accidental Complexity" of the code and make it easier for different engineers to read, maintain, and review.

C) Because the compiler requires specific indentation to work.

D) To prevent users from seeing the source code.

> Coding standards ensure that code looks consistent regardless of who wrote it. This is vital for long-term maintainability, as the person reading the code later is often not the person who wrote it.

Q4. (Comparison) What is the difference between **Internal Documentation** and **External Documentation**?

A) Internal documentation consists of comments and naming conventions inside the source code; External documentation includes SRS documents, user manuals, and API guides.

B) Internal is for users; External is for developers.

C) Internal documentation is written in C++; External is written in Java.

D) There is no difference; they are both stored in the database.

> Internal documentation is meant for the engineers maintaining the code. External documentation provides high-level context and instructions for users or other systems interacting with the software.

Q5. (Scenario-Based) A team decides to skip writing unit tests and ignores code reviews just to hit a very tight release deadline. They know they will have to go back and fix these issues later. This "cost of rework" is called:

A) Capital Expenditure

B) Scope Creep

C) Technical Debt

D) Refactoring

> Technical debt is the interest paid on "quick and dirty" coding. While it allows for a faster release now, it makes future changes much more difficult and expensive.

Q6. (Code/Console Logic) During a **Code Review**, a senior developer leaves the following comment on a junior's code. What is the senior developer suggesting?

pull-request-comment

```
"This nested if-else structure is 4 levels deep.
Please apply a 'Guard Clause' to flatten the logic."
```

A) Add more comments to explain the nesting.

B) Delete the entire function.

C) Refactor the code to handle edge cases/errors early and return, reducing the complexity of the main logic path.

D) Encrypt the if-else statements.

> A Guard Clause is a refactoring technique where you check for invalid conditions at the top of a function and return early. This "flattens" the code and makes it much more readable than deep nesting.

Q7. (Conceptual) What is the primary characteristic of **Self-Documenting Code**?

A) It uses an AI to write comments automatically.

B) It contains a 100-page README file.

C) It uses meaningful variable and function names (e.g., calculate_total_tax) that make the logic clear without needing extra comments.

D) It is written in a language that looks like plain English.

> The goal of professional implementation is to write code that is so clear that comments are almost unnecessary for understanding the "What" and "How."

Q8. (Comparison) In the context of implementation, what is the difference between **Logic Errors** and **Syntax Errors**?

A) Logic errors prevent the program from compiling; Syntax errors occur at runtime.

B) Syntax errors are violations of the language rules (caught by the compiler); Logic errors are mistakes in the program's reasoning (found during testing).

C) Syntax errors are for Python; Logic errors are for Java.

D) There is no difference.

> A program with a syntax error won't run. A program with a logic error runs perfectly but produces the wrong result (e.g., adding two numbers instead of multiplying them).

Q9. (Scenario-Based) An engineer is told that their implementation must be "Portable." This means the code should:

A) Be small enough to fit on a USB drive.

B) Use only one single file.

C) Be able to run on different operating systems or hardware platforms with minimal changes.

D) Be written in a language that supports pointers.

> Portability is a non-functional requirement often satisfied during the implementation phase by avoiding platform-specific libraries or hardcoded file paths.

Q10. (Conceptual) Which of the following describes the practice of **Pair Programming** during the implementation phase?

A) Two programmers writing two different features simultaneously.

B) One programmer writing code and the other writing documentation.

C) Two programmers working together at one computer; one (Driver) writes code while the other (Navigator) reviews and thinks ahead.

D) A programmer and a customer working together to design the UI.

> Pair programming is a collaborative implementation technique that results in fewer bugs and higher quality code through continuous real-time peer review.

Deep Dive: The Cost of Global Variables

One of the most fundamental implementation rules is **Avoid Global Variables**. While they seem easy to use, they introduce two major engineering risks:

- **Hidden Dependencies:** If Function A and Function B both modify `GlobalVarX`, they are tightly coupled. Changing Function A might unexpectedly break Function B.
- **Concurrency Risks:** In multi-threaded applications, multiple threads trying to write to a global variable simultaneously lead to "Race Conditions," which are among the hardest bugs to debug in software engineering.

Professional implementation favors **Encapsulation** and **Dependency Injection** to pass data safely between modules.

Stable Web Resources

- Wikipedia: Software Implementation
- Google: Coding Style Guides (Industry Standard)

Quick Revision Cheat Sheet

- **Defensive Programming:** Trust no input; check everything.
- **Technical Debt:** Quick fix now = Hard work later.
- **Refactoring:** Change structure, keep behavior.
- **Style Guides:** Ensure team consistency.
- **Unit Testing:** The first line of defense during implementation.
- **Syntax Error:** Grammar mistake. **Logic Error:** Reasoning mistake.

END OF SUBTOPIC 09 — MOVING TO SUBTOPIC 10

Subtopic 10: Software Testing (20 MCQs)

Quick Notes

- **Verification vs. Validation:** Verification asks "Are we building the product right?" (process-focused). Validation asks "Are we building the right product?" (meeting user needs).
- **Testing Levels:**
  - *Unit Testing:* Individual components/functions.
  - *Integration Testing:* Interaction between modules.
  - *System Testing:* The entire integrated system.
  - *Acceptance Testing (UAT):* Validation against user requirements.
- **Black-Box Testing:** Testing based on specifications without knowing internal code.
- **White-Box (Structural) Testing:** Testing based on internal logic, paths, and code structure.
- **Regression Testing:** Re-running old tests to ensure new changes haven't broken existing functionality.
- **Test Case:** Includes Input, Execution Condition, and **Expected Outcome**.

Common Traps & Confusions

- **Testing vs. Debugging:** Testing is finding bugs. Debugging is identifying the cause and fixing them. Developers debug; testers test.
- **The Exhaustive Testing Fallacy:** It is impossible to test all possible inputs and paths. We use strategies like Equivalence Partitioning to choose the most effective tests.
- **Alpha vs. Beta:** Alpha testing is done by internal staff at the developer's site. Beta testing is done by real users in their own environment.

MCQs

Q1. (Scenario-Based) A bank releases a new version of its mobile app to fix a small login bug. After the update, users find that they can no longer transfer money between accounts—a feature that worked perfectly before. What type of testing was skipped or performed inadequately?

A) Unit Testing

B) Stress Testing

C) Regression Testing

D) Alpha Testing

> Regression testing is specifically designed to ensure that changes (like bug fixes) do not negatively impact existing, previously working features.

Q2. (Calculation) A software component accepts an integer input representing a user's age, which must be between 18 and 65 (inclusive). Using **Boundary Value Analysis**, which set of values represents the best test inputs?

A) 20, 30, 40, 50

B) 0, 18, 65, 100

C) 17, 18, 19, 64, 65, 66

D) 18, 65

> Boundary Value Analysis focuses on the edges: (Min-1, Min, Min+1) and (Max-1, Max, Max+1). Errors most frequently occur at these transition points.

Q3. (Code/Console Logic) A developer is analyzing a function's **Control Flow Graph**. Based on the logic below, what is the minimum number of test cases required to achieve 100% **Statement Coverage**?

coverage_logic.cpp

```cpp
void checkAccess(bool isAdmin, bool isActive) {
    if (isAdmin) {
        logAdminAccess();
    }
    if (isActive) {
        grantAccess();
    }
```

```
}
```

A) 1

B) 1 (if both inputs are True)

C) 2

D) 4

> To achieve statement coverage, every line must execute at least once. If we provide (isAdmin=True, isActive=True), both 'if' blocks execute, covering every line in a single run. (Note: This does NOT achieve 100% Path or Branch coverage).

Q4. (Conceptual) Which testing technique is most effective for discovering **Logic Errors** and "Dead Code" that can never be executed?

A) Black-Box Testing

B) White-Box Testing

C) Acceptance Testing

D) Load Testing

> White-box testing looks inside the code structure. It uses techniques like basis path testing to ensure all logical branches and paths are reachable and correct.

Q5. (Scenario-Based) A software product is finished and is currently being tested by a selected group of end-users at their own locations before the general public release. This is known as:

A) Alpha Testing

B) Beta Testing

C) Integration Testing

D) Unit Testing

> Beta testing is external acceptance testing where real users use the software in a real-world setting to find final bugs and usability issues.

Q6. (Comparison) What is the primary difference between **Static Testing** and **Dynamic Testing**?

A) Static is for C++; Dynamic is for Java.

B) Static testing involves inspecting code or documents without executing the software; Dynamic testing involves running the program with test data.

C) Dynamic testing is always cheaper.

D) Static testing is only for the UI.

> Static testing (reviews, walkthroughs, inspections) finds errors before the code is ever run. Dynamic testing validates the behavior of the software during execution.

Q7. (Code/Console Logic) A QA engineer is looking at an **Automation Report**. Based on the console output, which testing category is being executed?

test-runner --verbose

```
[RUN] Testing 'AuthModule' + 'DatabaseModule'
[LOG] Checking data flow from login -> user_session_table
[PASS] Session record created successfully.
```

A) Unit Testing

B) Integration Testing

C) Stress Testing

D) Security Testing

> The log shows two distinct modules (Auth and Database) being tested together to see how they interact. This is the definition of integration testing.

Q8. (Conceptual) In the context of **Verification**, what is a "Walkthrough"?

A) A physical walk through the data center.

B) An informal meeting where the author of a document or code explains it to a group of peers to find errors.

C) Running the software step-by-step in a debugger.

D) Delivering the software to the client's office.

> A walkthrough is a static testing technique used for verification. Its primary goal is to share knowledge and find defects through peer review.

Q9. (Calculation) A tester uses **Equivalence Partitioning** for a field that accepts marks from 0 to 100. How many "Invalid" partitions are there?

A) 1

B) 2 (Less than 0, and Greater than 100)

C) 3

D) 0

> Equivalence partitioning divides input into valid and invalid groups. Partition 1: marks < 0 (Invalid); Partition 2: 0-100 (Valid); Partition 3: marks > 100 (Invalid).

Q10. (Scenario-Based) A gaming company wants to see if their server will crash if 100,000 players log in at the exact same second. This is an example of:

A) Functional Testing

B) Regression Testing

C) Stress Testing

D) Unit Testing

> Stress testing pushes a system beyond its specified limits to see how it fails and how it recovers.

Q11. (Comparison) Compare the **Top-Down** and **Bottom-Up** integration testing strategies. What is the main requirement for **Top-Down** testing?

A) Drivers

B) Stubs

C) A finished database

D) At least 50 developers

> In Top-Down testing, high-level modules are tested before low-level ones. "Stubs" are used as dummy replacements for the low-level modules that haven't been built yet. (Bottom-Up uses "Drivers").

Q12. (Conceptual) What does the **"Pesticide Paradox"** in software testing suggest?

A) Software contains bugs like plants contain pests.

B) If the same set of tests is repeated over and over, eventually the same set of test cases will no longer find new bugs.

C) Testing should only be done at night.

D) Bugs are caused by hardware heat.

> This principle highlights the need to constantly review and update test cases to find new defects as the software evolves.

Q13. (Scenario-Based) You are testing a navigation system. You verify that the GPS coordinates are calculated correctly (Verification). You then give the system to a pilot who says "the font is too small to read while flying." This feedback relates to:

A) Verification Failure

B) Validation Failure

C) Syntax Error

D) Integration Error

> The pilot is saying the product does not meet their actual needs in the work environment, which is a validation issue, even if the math (verification) was perfect.

Q14. (Calculation) A function has a **Cyclomatic Complexity** of 5. What does this number technically tell the tester?

A) The function will take 5 seconds to run.

B) There are exactly 5 independent linear paths through the code that must be tested.

C) There are 5 variables in the function.

D) The function is written in 5 different languages.

> Cyclomatic complexity provides an upper bound for the number of test cases required to achieve 100% branch/path coverage.

Q15. (Code/Console Logic) Which testing tool or framework is commonly used for **Unit Testing** in the Java ecosystem, as indicated by the following metadata?

pom.xml

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
```

```
</dependency>
```

A) Selenium

B) Jenkins

C) JUnit

D) Postman

> JUnit is the industry-standard unit testing framework for Java. It allows developers to write and run repeatable automated tests.

Q16. (Conceptual) In software testing, what is **"Oracle"**?

A) A large database company.

B) A mechanism (manual or automated) for determining whether a test has passed or failed by comparing actual results to expected ones.

C) A bug that is impossible to find.

D) The lead developer of the project.

> A Test Oracle is anything that provides the "correct" answer to a test case. Without an oracle, we cannot know if a test result is right or wrong.

Q17. (Scenario-Based) A tester is performing **Exploratory Testing**. This means they are:

A) Following a strict, pre-written script of 100 steps.

B) Learning the system, designing test cases, and executing them simultaneously without a predefined plan.

C) Using a robot to click buttons randomly.

D) Reading the source code to find errors.

> Exploratory testing relies on the tester's skill and intuition to "explore" the system and find bugs that scripted tests might miss.

Q18. (Comparison) What is the difference between **Smoke Testing** and **Sanity Testing**?

A) Smoke testing checks the critical core functionality of a build; Sanity testing checks if specific bugs have been fixed and if a particular component works as expected.

B) Smoke testing is for hardware; Sanity is for software.

C) Smoke testing is done by users; Sanity is done by developers.

D) There is no difference.

> Smoke testing is a broad "shallow" test to see if the build is stable enough to start testing. Sanity testing is a narrower, deeper test on a specific change.

Q19. (Conceptual) What is the primary objective of **Performance Testing**?

A) To find syntax errors in the code.

B) To ensure the UI is pretty.

C) To measure the system's responsiveness, stability, and resource usage under a specific workload.

D) To make the code run faster automatically.

> Performance testing checks non-functional requirements like speed (latency) and throughput to ensure the system can handle its intended load.

Q20. (Scenario-Based) A tester discovers a bug where the "Submit" button works when clicked once, but if clicked ten times rapidly, it creates duplicate orders. This is a failure in:

A) Reliability

B) Portability

C) Robustness (Concurrency Handling)

D) Maintainability

> Robustness is the ability of software to cope with errors during execution and cope with "stressful" or unusual user behavior (like rapid clicking).

Deep Dive: Cyclomatic Complexity (McCabe's Metric)

To measure the complexity of a program and determine the minimum number of tests needed for white-box coverage, we use the formula: `M = E - N + 2P`

- **E:** Number of edges in the control flow graph.
- **N:** Number of nodes in the graph.
- **P:** Number of connected components (usually 1 for a single function).

Example: A simple function with one `if-else` statement has 4 nodes (start, decision, two paths, end) and 4 edges. Complexity = `4 - 4 + 2(1) = 2`. This means you need exactly 2 test cases (one for True, one for False) to cover both branches.

Stable Web Resources

- Wikipedia: Software Testing
- ISTQB: International Software Testing Qualifications Board

Quick Revision Cheat Sheet

- **Unit:** Function level. **Integration:** Interface level. **System:** Full app level.
- **Black-box:** No code view. **White-box:** Full code view.
- **Equivalence Partitioning:** Grouping similar inputs to save time.
- **Boundary Value:** Testing the extreme edges (-1, 0, +1).
- **Regression:** "Did I break what was already working?"
- **Verification:** Build it right. **Validation:** Build the right thing.

END OF SUBTOPIC 10 — MOVING TO SUBTOPIC 11

Subtopic 11: Software Maintenance & Evolution (10 MCQs)

Quick Notes

- **Software Maintenance:** The process of modifying a software system after delivery to correct faults, improve performance, or adapt it to a changed environment.
- **Types of Maintenance:**
  - *Corrective:* Fixing bugs found by users.

- *Adaptive:* Modifying software to work in a new environment (e.g., new OS, new hardware).
- *Perfective:* Adding new features or improving performance/maintainability.
- *Preventive:* Refactoring code to prevent future problems (Software Re-engineering).

- **Lehman's Laws:** A set of observations on how software evolves. Key laws include *Continuing Change* (software must change or become useless) and *Increasing Complexity* (evolution causes structure to degrade).
- **Legacy Systems:** Old systems that are still essential to an organization but difficult and expensive to maintain.
- **Reverse Engineering:** Analyzing a system to create a representation of it at a higher level of abstraction (e.g., generating a design diagram from source code).

Common Traps & Confusions

- **Maintenance vs. Development:** Maintenance is often seen as "lesser work," but it typically accounts for 60% to 90% of a software's total lifetime cost.
- **Re-engineering vs. Replacement:** Re-engineering keeps the same functionality but improves the internal "health" of the code. Replacement is building a new system from scratch.
- **Perfective vs. Adaptive:** If you are adding a feature because the *user* wants it, it is Perfective. If you are changing the code because the *browser/OS* updated, it is Adaptive.

MCQs

Q1. (Scenario-Based) A company’s internal HR system was built for Windows 7. After the company upgrades all laptops to Windows 11, the software fails to launch. The development team is tasked with updating the system calls to be compatible with the new OS. This is an example of:

A) Corrective Maintenance

B) Adaptive Maintenance

C) Perfective Maintenance

D) Preventive Maintenance

> Adaptive maintenance is performed to make the software work in a changed environment (new OS, hardware, or third-party APIs) without changing the core functionality.

Q2. (Conceptual) According to **Lehman's Law of Increasing Complexity**, what happens to a software system as it evolves through repeated maintenance cycles?

A) The software becomes simpler and easier to understand.

B) The number of lines of code decreases automatically.

C) The internal structure of the software degrades and becomes more complex unless active work is done to maintain it.

D) The software becomes immune to bugs.

> Lehman observed that as systems evolve, their "entropy" increases. New features are often "patched" in, which makes the original clean design messy and harder to maintain over time.

Q3. (Code/Console Logic) A lead engineer is reviewing a system maintenance log. Based on the `diff` output below, what type of maintenance activity was performed?

```
git-diff --patch

- if (input == null) return;
- process(input);
+ if (input == null || input.trim().isEmpty()) {
+     log.error("Invalid input received");
+     throw new IllegalArgumentException();
+ }
+ process(input);
```

A) Corrective Maintenance

B) Adaptive Maintenance

C) Perfective Maintenance

D) Software Evolution

> The patch shows the addition of input validation and error handling to prevent a crash or unexpected behavior. Fixing faults to ensure the software works correctly is Corrective Maintenance.

Q4. (Scenario-Based) A video streaming platform notices that their "Recommendation Engine" is slow. They decide to rewrite the underlying sorting algorithm from O(N^2) to O(N log N) to make the page load faster, though the recommendations stay exactly the same. This is:

A) Corrective Maintenance
B) Preventive Maintenance
C) Perfective Maintenance
D) Reverse Engineering

> Perfective maintenance includes improvements to the software's non-functional attributes, such as performance (speed) or efficiency, even if no new user features are added.

Q5. (Comparison) What is the primary difference between **Software Re-engineering** and **Forward Engineering**?

A) Forward engineering is for bugs; Re-engineering is for UI.
B) Forward engineering starts from requirements and moves to implementation; Re-engineering starts from an existing system to rebuild it with higher quality.
C) Re-engineering is only for Java; Forward is for C++.
D) Forward engineering is faster than Re-engineering.

> Forward engineering is the traditional "start to finish" process. Re-engineering is a "circular" process where you take an existing, messy system, analyze it, and rebuild it into a cleaner, more maintainable version.

Q6. (Conceptual) In the context of **Legacy Systems**, what is "Software Sclerosis"?

A) A virus that attacks servers.
B) The state where a system becomes so difficult to modify that every change causes more problems than it solves.
C) The process of deleting old documentation.
D) High-speed execution of legacy code.

> Software Sclerosis (or aging) occurs when constant maintenance has degraded the architecture so much that the code is fragile and "hardened" against any further change.

Q7. (Scenario-Based) An organization has a 20-year-old COBOL system. They use a tool that reads the COBOL source code and automatically generates **UML Sequence Diagrams** to help new developers understand the logic. This is an example of:

A) Code Refactoring

B) Reverse Engineering

C) Data Migration

D) Regression Testing

> Reverse Engineering is the process of moving "backward" from lower levels of abstraction (code) to higher levels (design/requirements) to aid in understanding.

Q8. (Calculation/Comparison) A software manager is comparing the cost of **Maintenance** versus **Replacement**. If maintenance costs $10,000/year and replacing the system costs $150,000 but reduces maintenance to $1,000/year, after how many years is replacement more cost-effective?

A) 5 years

B) 10 years

C) 17 years (approx)

D) 25 years

> Savings per year = 10,000 - 1,000 = $9,000. Break-even = Cost of Replacement / Savings = 150,000 / 9,000 = 16.66 years. Management must decide if the system will live long enough to justify the replacement cost.

Q9. (Conceptual) Which of **Lehman's Laws** states that "the functional content of a system must be continually increased to maintain user satisfaction over its lifetime"?

A) Continuing Change

B) Continuing Growth

C) Conservation of Familiarity

D) Self Regulation

> Lehman's Law of Continuing Growth suggests that software must not just be maintained (fixed/adapted) but must also grow by adding new features to stay relevant in a competitive market.

Q10. (Code/Console Logic) A developer is looking at a **Refactoring** task. Which design metric is primarily being improved by the change shown in this console output?

refactoring-log.txt

```
# Before: One large function (200 lines) doing Auth and Logging.
# After: Split into 'authenticateUser()' and 'logEvent()'.
```

A) Coupling

B) Cohesion

C) Portability

D) Efficiency

> By splitting a multi-tasking function into smaller, specialized functions that each do "one thing," the developer is increasing the **Cohesion** of the code, which is a key goal of Preventive Maintenance.

Deep Dive: Lehman's Laws of Software Evolution

Manny Lehman and Les Belady identified 8 laws after studying the evolution of IBM's OS/360. The three most critical for engineers are:

1. **Law of Continuing Change:** An E-type (Real-world) system must be continually adapted or it becomes progressively less satisfactory.
2. **Law of Increasing Complexity:** As a system evolves, its complexity increases unless work is done to maintain or reduce it (Refactoring).
3. **Law of Declining Quality:** The quality of a system will appear to be declining unless it is adapted to changes in its operational environment.

This explains why "finished" software is never really finished until it is retired.

Stable Web Resources

- Wikipedia: Lehman's Laws
- GeeksforGeeks: Maintenance Process

Quick Revision Cheat Sheet

- **Corrective:** Fixing bugs.
- **Adaptive:** New OS/Hardware.
- **Perfective:** New Features/Better Speed.
- **Preventive:** Refactoring/Cleaning code.
- **60-90%:** The cost of maintenance in the total SDLC.
- **Evolution:** Software is a living entity that must grow or die.
- **Reverse Engineering:** Code → Design.

END OF SUBTOPIC 11 — MOVING TO SUBTOPIC 12

Subtopic 12: Software Quality Assurance (10 MCQs)

Quick Notes

- **Software Quality Assurance (SQA):** A systematic pattern of actions that are necessary to provide high confidence that the software conforms to established requirements and standards.
- **QA vs. QC:**
  - *QA (Assurance):* Process-oriented. Prevents defects (e.g., audits, training).
  - *QC (Control):* Product-oriented. Detects defects (e.g., testing, reviews).
- **ISO/IEC 25010:** The modern standard for Software Quality requirements. Key attributes: Functional Suitability, Reliability, Performance Efficiency, Usability, Security, Compatibility, Maintainability, and Portability.
- **CMMI (Capability Maturity Model Integration):** A process level improvement training and appraisal program.
  - Level 1: Initial (Chaotic).
  - Level 2: Managed (Project level).
  - Level 3: Defined (Organization level).
  - Level 4: Quantitatively Managed.

    - Level 5: Optimizing (Continuous improvement).
- **Cost of Quality:** Prevention costs (training), Appraisal costs (testing), and Failure costs (internal bugs vs. external crashes).

Common Traps & Confusions

- **Quality is not "Bling":** Quality isn't about having the most features; it's about meeting the *specified* requirements accurately.
- **Testing is NOT QA:** Testing is a subset of Quality Control. QA involves the entire development process, including how requirements are gathered and how code is reviewed.
- **Zero Defects Myth:** In complex software engineering, "Quality" often means reducing risk to an *acceptable* level, as achieving 100% bug-free code is mathematically and economically impossible.

MCQs

Q1. (Scenario-Based) A software manager introduces a mandatory peer-review process for all design documents before coding begins. They also organize a workshop to train developers on secure coding standards. These activities are best described as:

A) Software Quality Control (SQC)
B) Software Quality Assurance (SQA)
C) Software Testing
D) Debugging

> SQA is process-oriented and focuses on prevention. By improving the process (reviews and training) rather than just checking the final product, the manager is performing Quality Assurance.

Q2. (Conceptual) According to the **ISO/IEC 25010** quality model, which attribute describes the degree to which a system or component can be effectively and efficiently modified by the developers?

A) Reliability
B) Portability

C) Maintainability

D) Functional Suitability

> Maintainability includes sub-characteristics like Modularity, Reusability, Analysability, Modifiability, and Testability. It focuses on the internal quality that allows the software to evolve.

Q3. (Code/Console Logic) A developer is looking at a **Static Analysis (Linting)** report. Which Quality Attribute is primarily being targeted by the warning shown in the console?

sonarqube-report.log

```
[MAJOR] Cognitive Complexity of function 'processOrder()' is 28.
[HINT]  Recommended maximum is 15.
[HINT]  Extract logic into smaller, testable methods.
```

A) Security

B) Efficiency

C) Maintainability

D) Portability

> High cognitive complexity means the code is difficult to read and understand. Reducing complexity directly improves "Maintainability" and "Analysability" for future developers.

Q4. (Comparison) Compare **Internal Failure Costs** and **External Failure Costs**. Which is generally more expensive for a software organization?

A) Internal Failure Costs, because the developers have to be paid overtime to fix bugs.

B) External Failure Costs, because they involve lost customers, damage to brand reputation, and potential legal liabilities after the product is released.

C) They are exactly the same.

D) Internal failure costs, because they require re-running the unit tests.

> The "Rule of 10" suggests that a bug found by a user (External) costs 10x to 100x more to fix than a bug found during development (Internal) due to the overhead of patch deployment and business impact.

Q5. (Scenario-Based) An organization has documented all its software processes, and every project team is now required to follow the same standard set of procedures for development and maintenance. According to the **CMMI** model, this organization is likely at:

A) Level 1 (Initial)

B) Level 2 (Managed)

C) Level 3 (Defined)

D) Level 5 (Optimizing)

> At Level 2, processes are managed at a project level. At Level 3 (Defined), processes are standardized across the *entire organization*, ensuring consistency and predictability.

Q6. (Conceptual) What is the primary purpose of a **Quality Audit** in Software Engineering?

A) To find and fix syntax errors in the source code.

B) To provide an independent evaluation of whether the project is following the defined processes, standards, and regulations.

C) To calculate the final price of the software for the customer.

D) To replace the unit testing phase.

> An audit is a high-level QA activity. It checks if the "Process" is being respected. For example, "Did the team actually perform the required security reviews before deployment?"

Q7. (Code/Console Logic) A Lead QA Engineer calculates the **Defect Density** of two modules. Based on the logic below, which module is of "Higher Quality" from a defect perspective?

qa-metrics.py

```
# Module A: 10 bugs in 1,000 lines of code (KLOC)
# Module B: 50 bugs in 10,000 lines of code (KLOC)

density_a = 10 / 1
density_b = 50 / 10
```

A) Module A (10 bugs per KLOC)

B) Module B (5 bugs per KLOC)

C) They are equal quality.

D) Module A is better because it has fewer total bugs.

> Quality is relative to size. Module A has 10 defects per 1,000 lines. Module B has only 5 defects per 1,000 lines. Even though B has more *total* bugs, it is statistically more reliable per unit of code.

Q8. (Scenario-Based) A mission-critical system for a power plant has a required **Availability** of "Four Nines" (99.99%). This means the system can be down for at most how long per year?

A) 1 hour

B) 52 minutes (approx)

C) 3 days

D) 8 hours

> Availability is a quality metric. 99.99% means the system is down for 0.01% of the time. 365 days * 24 hours * 60 minutes * 0.0001 = 52.56 minutes. High availability is a key SQA goal.

Q9. (Comparison) Compare **Product Quality** and **Process Quality**. Which of the following statements is a fundamental assumption of SQA?

A) Product quality is independent of process quality.

B) A high-quality process will naturally lead to high-quality software products.

C) Process quality is only important for small teams.

D) Automated tools can replace the need for process quality.

> The core philosophy of QA (and TQM) is that if you follow a disciplined, tested, and documented process, the output (the product) will consistently meet high standards.

Q10. (Conceptual) In SQA, what does the **"Pareto Principle" (80/20 Rule)** imply regarding software defects?

A) 80% of the code will have 20% of the bugs.

B) 80% of the defects are likely to be found in 20% of the software modules.

C) 80% of the developers write 20% of the code.

D) 80% of the budget is spent on 20% of the hardware.

> In SQA, the Pareto Principle helps focus effort. It suggests that bugs are not distributed evenly; instead, a few complex or poorly written modules usually contain the vast majority of errors.

Deep Dive: Total Quality Management (TQM)

TQM in software engineering is a culture of quality that involves every stakeholder. It is built on four pillars:

1. **Customer Focus:** Quality is ultimately defined by the user's satisfaction.
2. **Continuous Improvement (Kaizen):** The process is never "perfect"; teams should constantly find small ways to reduce waste and defects.
3. **Employee Empowerment:** Developers and testers should have the authority to "stop the line" if a major quality issue is found.
4. **Data-Driven Decisions:** Use metrics like Mean Time Between Failures (MTBF) and Defect Removal Efficiency (DRE) rather than "gut feelings" to judge progress.

Stable Web Resources

- Wikipedia: Software Quality Assurance
- ISO 25010: Software Quality Model

Quick Revision Cheat Sheet

- **QA:** Process focus (Preventive). **QC:** Product focus (Detective).
- **ISO 2510:** Security, Reliability, Performance, etc.
- **CMMI Level 5:** The peak of process maturity (Continuous improvement).
- **Defect Density:** Bugs per KLOC (Size-relative quality).
- **Pareto Principle:** Focus on the 20% of code that has 80% of bugs.
- **Appraisal Cost:** Cost of checking for quality (Testing).
- **Prevention Cost:** Cost of ensuring quality (Training/Standards).

END OF SUBTOPIC 12 — MOVING TO SUBTOPIC 13

Subtopic 13: Software Metrics & Measurement (10 MCQs)

Quick Notes

- **Software Metric:** A quantitative measure of the degree to which a system, component, or process possesses a given attribute.
- **Metric Types:**
  - *Product Metrics:* Describe characteristics of the software (e.g., size, complexity, reliability).
  - *Process Metrics:* Used to improve software development and maintenance (e.g., defect removal efficiency).
  - *Project Metrics:* Describe project execution (e.g., cost, schedule, staffing).
- **Measurement Types:**
  - *Direct:* Measured directly (e.g., Lines of Code, execution speed, memory used).
  - *Indirect:* Derived from other measures (e.g., Quality, Functionality, Maintainability).
- **Halstead Metrics:** Based on the count of operators and operands to estimate program volume and effort.
- **GQM Paradigm:** Goal → Question → Metric. A framework for choosing the right metrics based on business objectives.

Common Traps & Confusions

- **LOC is NOT Productivity:** A programmer who writes 100 lines of efficient code is often better than one who writes 1,000 lines of messy code. Metrics must be context-aware.
- **Internal vs. External Attributes:** Internal attributes (like complexity) can be measured by looking at the code. External attributes (like usability or reliability) can only be measured when the code is running.

- **Stale Metrics:** Collecting data without using it for decision-making is "vanity metrics" and provides no engineering value.

MCQs

Q1. (Comparison) A manager wants to compare the **Productivity** of a C++ team and a Python team. Why is **Lines of Code (LOC)** a poor direct metric for this comparison?

A) Because Python doesn't use lines of code.
B) Because LOC is language-dependent; Python is more expressive and achieves the same functionality with fewer lines than C++, penalizing the Python team.
C) Because C++ is an older language.
D) Because LOC can only be measured after the project is finished.

> LOC is a "size-oriented" metric that varies wildly between high-level and low-level languages. Using it as a productivity measure ignores the efficiency and expressiveness of modern languages.

Q2. (Conceptual) Which of the following is an example of an **Indirect Measure** of software quality?

A) Total number of pages in the documentation.
B) Amount of RAM consumed by the application.
C) Maintainability, calculated by measuring the time taken to fix a bug and the complexity of the code.
D) Total number of lines in the source code.

> Indirect measures are "abstract" qualities that cannot be measured directly. Maintainability is an external attribute derived from direct measures like "Mean Time To Repair" (MTTR).

Q3. (Scenario-Based) An organization wants to measure how effective their testing process is at finding bugs before the product reaches the customer. They should use which metric?

A) Defect Density
B) Defect Removal Efficiency (DRE)
C) Cyclomatic Complexity
D) Response Time

> DRE = (Errors found during development / Errors found during development + Errors found by user). A high DRE indicates a very high-quality QA process that filters out bugs early.

Q4. (Code/Console Logic) A Lead Developer is automating the calculation of **Defect Removal Efficiency (DRE)** for the last release. Based on the logic below, what is the DRE for this project?

metrics_calc.py

```
bugs_found_internally = 80
bugs_reported_by_users = 20

dre = bugs_found_internally / (bugs_found_internally + bugs_reported_by
print(f"DRE: {dre * 100}%")
```

A) 100%

B) 80%

C) 20%

D) 400%

> DRE = 80 / (80 + 20) = 80 / 100 = 0.8. Multiplying by 100 gives 80%. This means the internal team caught 80% of the total bugs that existed in the system.

Q5. (Conceptual) In the **Goal-Question-Metric (GQM)** approach, what is the purpose of the "Question" layer?

A) To quiz the developers on their knowledge.

B) To find out why the software is crashing.

C) To define specific queries that characterize the goal and can be answered by data.

D) To ask the customer for more money.

> GQM is a top-down approach. If the Goal is "Improve Reliability," the Question might be "How often is the system failing?" and the Metric would be "Mean Time Between Failures (MTBF)."

Q4. (Scenario-Based) A project manager uses **Mean Time to Failure (MTTF)** to evaluate a system. This metric is primarily a measure of:

A) Portability

B) Efficiency

C) Reliability

D) Scalability

> MTTF measures the average time a system runs without failing. It is the core quantitative metric for assessing software reliability.

Q7. (Comparison) Compare **Token-based metrics** (like Halstead's) with **Structure-based metrics** (like McCabe's). What is the main focus of **McCabe's Cyclomatic Complexity**?

A) The number of variables in the code.

B) The logical branching and decision points in the code.

C) The length of the user manual.

D) The number of lines of comments.

> McCabe’s complexity focuses on the control flow of the program. It counts decision points (if, while, for) to determine the number of independent paths, which indicates how difficult the code is to test and maintain.

Q8. (Calculation) A project has 10 **Function Points (FP)**. If the organization’s historical data shows they can produce 5 FP per Person-Month, what is the estimated **Effort** for this project?

A) 50 Person-Months

B) 0.5 Person-Months

C) 2 Person-Months

D) 15 Person-Months

> Effort = Total Size / Productivity. So, 10 FP / 5 FP per PM = 2 Person-Months.

Q9. (Conceptual) What does **"Fan-in"** represent in a structural software metric?

A) The number of users logged into the system.

B) The number of other components that call or use a specific component.

C) The number of files in the project folder.

D) The speed of the cooling fan in the server.

> High Fan-in means a module is heavily reused, which is generally good. However, if that module changes, it has a "Ripple Effect" on many other parts of the system.

Q10. (Scenario-Based) A company wants to measure the "Productivity" of its support team which handles maintenance. Which metric is most appropriate for this "Process" evaluation?

A) Total KLOC of the system.

B) Number of tickets resolved per week.

C) Cyclomatic complexity of the legacy code.

D) Number of `private` variables in the classes.

> Process metrics focus on the activities of the people. Throughput (tickets per week) measures the efficiency of the maintenance process, whereas KLOC and complexity are product metrics.

Deep Dive: The Software Maturity Index (SMI)

As a system evolves, we need to know if it is becoming more stable or more fragile. The **Software Maturity Index (SMI)** is used for this:

```
SMI = [ Mt - (Fa + Fc + Fd) ] / Mt
```

- **Mt:** Number of modules in the current release.
- **Fa:** Number of modules added.
- **Fc:** Number of modules changed.
- **Fd:** Number of modules deleted from the previous release.

As the SMI approaches **1.0**, the software is considered stable. A low SMI indicates a system in high flux (likely a legacy system undergoing heavy, risky changes).

Stable Web Resources

- Wikipedia: Software Metrics
- TutorialsPoint: Complexity Metrics

Quick Revision Cheat Sheet

- **Product Metric:** What the software IS (Size, Complexity).
- **Process Metric:** How the software is MADE (Speed, Bug rate).
- **DRE:** Filter quality (Internal bugs / Total bugs).
- **MTBF / MTTF:** Reliability (Time between crashes).
- **Cyclomatic Complexity:** Decision points + 1.
- **Fan-out:** Number of modules called BY a module.
- **Fan-in:** Number of modules calling A module.

END OF SUBTOPIC 13 — MOVING TO SUBTOPIC 14

Subtopic 14: Software Configuration Management (10 MCQs)

Quick Notes

- **SCM (Software Configuration Management):** The discipline of tracking and controlling changes in the software development lifecycle.
- **Configuration Items (CIs):** The "things" we track—source code files, design documents, test plans, build scripts, and external libraries.
- **Baseline:** A stable version of a software configuration item that has been reviewed and agreed upon; it serves as a basis for further development.
- **Version Control Systems:**
  - *Centralized (CVCS):* One server, all clients check in/out (e.g., SVN).
  - *Distributed (DVCS):* Every developer has a full clone of the repository (e.g., Git).
- **Change Control:** The process of managing change requests (CRs) through assessment, approval, and implementation.

Common Traps & Confusions

- **SCM vs. Backup:** SCM tracks the *history* and *evolution* of changes. Backups are just a point-in-time copy. You need both.
- **Baseline vs. Release:** A baseline is a stable milestone in development. A release is the final, production-ready version provided to the end-user.
- **Git Merge vs. Rebase:** Merge preserves history; Rebase "rewrites" history to make it look linear. Both are correct, but they serve different workflow philosophies.

MCQs

Q1. (Scenario-Based) A team of developers accidentally overwrites each other's changes in the same source file because they are all saving directly to a shared network folder. What is the most basic SCM tool they should implement to solve this?

A) An automated build script.
B) A better project manager.
C) A Version Control System (like Git or SVN).
D) A shared database.

> Version Control is designed specifically to handle concurrent edits, resolve merge conflicts, and keep a history of who changed what and why.

Q2. (Conceptual) In SCM, what is a **"Baseline"**?

A) The initial cost of the project.
B) A stable version of a configuration item that has been formally reviewed and serves as a foundation for further development.
C) The set of computers used for testing.
D) A list of all developers on the project.

> Baselines provide a "known good state." If development goes off the rails, you can always revert to the baseline to regain stability.

Q3. (Comparison) What is the key advantage of a **Distributed Version Control System (DVCS)** over a Centralized one?

A) It uses less disk space on the developer's machine.

B) Each developer has a full backup of the repository, allowing work to continue offline and providing better resilience.

C) It is impossible to commit changes in a DVCS.

D) Centralized systems are always faster.

> In Git (DVCS), you don't need a network connection to commit or view history. Every clone is a complete repository, making the system highly resilient to server failures.

Q4. (Code-Tracing) A developer sees the following error in their terminal. What is the immediate reason for this?

```
git-terminal

$ git pull
...
CONFLICT (content): Merge conflict in main.py
Automatic merge failed; fix conflicts and then commit the result.
```

A) The network connection is down.

B) The developer doesn't have permission to edit the file.

C) The same lines of code were modified by two different developers, and Git cannot automatically decide which version to keep.

D) The file is read-only.

> Merge conflicts happen when the version control system cannot "auto-merge" because the changes overlap at the exact same location in the source code.

Q5. (Conceptual) What is the primary role of the **Change Control Board (CCB)**?

A) To write the source code for the project.

B) To review, evaluate, and approve or reject change requests based on impact, cost, and risk.

C) To test the final application before deployment.

D) To hire new developers.

> The CCB is a governance body. Its goal is to prevent uncoordinated changes that might destabilize the project or explode the budget.

Q6. (Conceptual) In SCM, what is the process of **Checking Out** a file vs. **Checking In**?

A) Checking out = Uploading; Checking in = Downloading.

B) Checking out = Retrieving/Locking for edit; Checking in = Submitting changes back to the repository.

C) Checking out = Deleting; Checking in = Creating.

D) They are the same thing.

> In centralized systems, checking out often locks a file so others can't edit it, ensuring that only one person works on a file at a time. Checking in submits the completed work.

Q7. (Conceptual) What is a **Branch** in a Version Control system?

A) A copy of the database.

B) An independent line of development that allows features to be worked on without affecting the main code base.

C) A backup of the hard drive.

D) A folder that cannot be deleted.

> Branching allows developers to work on new features or bug fixes in isolation ("feature branches") without contaminating the "stable" main branch until the code is reviewed and approved.

Q8. (Conceptual) Why is **"Tagging"** a version (e.g., `v1.0.0`) important in SCM?

A) To label the files for the operating system to find them.

B) To provide a permanent, immutable marker for a specific state of the repository, like a release.

C) To increase the security of the file.

D) It is an optional feature for cosmetics.

> Tags are used to mark releases. If a bug is reported in version 1.0, you can "checkout" the 1.0 tag to see exactly what the code looked like at that moment, even if the main branch has moved on significantly.

Q9. (Scenario-Based) An application has a configuration file `config.yaml` that contains database passwords. What is the standard security practice in SCM?

A) Commit the password file to Git so everyone has it.

B) Add the file to `.gitignore` and provide a `config.example.yaml` template instead.

C) Encrypt the password in the file before committing.

D) Commit it but only allow the lead dev to see it.

> Secrets (passwords/keys) should never be committed to version control. They should be stored in secure environment variables or secret management tools.

Q10. (Conceptual) What is a **"Head"** in the context of a Version Control Repository?

A) The server that hosts the Git repository.

B) A pointer that represents the most recent commit in the current branch.

C) The developer who created the repository.

D) The first line of the source code.

> "HEAD" is a special pointer that tells the VCS which commit you are currently looking at or working on. Usually, it points to the tip of your branch.

Deep Dive: Atomic Commits

An "Atomic Commit" is a fundamental principle of effective SCM. It means that **a commit should represent one, and only one, logical change.**

- **Good:** A commit that fixes a specific bug or adds one specific feature.
- **Bad:** A commit labeled "Update" that includes 10 unrelated bug fixes, a UI change, and a dependency upgrade.

Why this matters for Recovery: If a "Bad" commit introduces a bug, you can't revert just the bug fix—you have to revert everything in that giant commit. Atomic commits allow for surgical precision when things go wrong.

Stable Web Resources

- Git Documentation

- Wikipedia: Configuration Management

Quick Revision Cheat Sheet

- **CI:** Tracks history and changes.
- **Baseline:** The "Known Good" foundation.
- **Branch:** Parallel workstreams.
- **Merge Conflict:** Manual resolution of overlapping code changes.
- **Atomic:** One commit = One task.
- **.gitignore:** Don't commit secrets or build artifacts.

END OF SUBTOPIC 14 — MOVING TO SUBTOPIC 15

Subtopic 15: Software Risk Management (10 MCQs)

Quick Notes

- **Software Risk:** An uncertain event or condition that, if it occurs, has a positive or negative effect on a project's objectives (scope, schedule, cost, quality).
- **Risk Management Process:**
  - *Identification:* What can go wrong? (e.g., technical, project, or business risks).
  - *Analysis (Assessment):* How likely is it? (Probability) and How bad is it? (Impact).
  - *Planning (Mitigation):* What will we do about it? (Avoid, Transfer, Mitigate, Accept).
  - *Monitoring:* Tracking risks throughout the project.
- **Risk Exposure (RE):** Calculated as `Probability * Impact`.
- **Contingency Plan:** A "Plan B" that is executed only if the risk actually turns into an issue.

Common Traps & Confusions

- **Risk vs. Issue:** A *Risk* is in the future (it might happen). An *Issue* is in the present (it already happened). You manage risks; you resolve issues.
- **Mitigation vs. Contingency:** Mitigation is a proactive step to reduce probability/impact (e.g., training developers). Contingency is a reactive plan (e.g., if the project runs late, we cut feature X).
- **Acceptance:** Accepting a risk doesn't mean ignoring it; it means documenting it, budgeting for it, and acknowledging that the cost of preventing it is higher than the cost of the impact.

MCQs

Q1. (Calculation) A project team identifies a technical risk: "The new API might not be ready on time." The probability is 40% (0.4) and the estimated impact on the budget is $50,000. What is the **Risk Exposure (RE)**?

A) $10,000

B) $20,000

C) $50,000

D) $200,000

> Risk Exposure = Probability * Impact. $50,000 * 0.4 = $20,000. This value helps managers prioritize which risks to address first.

Q2. (Scenario-Based) Which of the following is an example of **Risk Transfer**?

A) Installing a fire alarm to prevent server room fires.

B) Buying insurance for the project or hiring a third-party vendor to assume the technical liability.

C) Ignoring the risk because it is unlikely.

D) Redoing the project plan to remove the dangerous feature.

> Risk Transfer involves shifting the financial or performance burden of the risk to a third party (like an insurance company or a specialized outsourcing firm).

Q3. (Comparison) Compare a **Project Risk** and a **Technical Risk**. Which of the following is a Project Risk?

A) The sorting algorithm might be too slow.

B) The database might deadlock under load.

C) The lead developer might resign unexpectedly.

D) The API might have security vulnerabilities.

> Technical risks (A, B, D) deal with the software's performance or implementation. Project risks (C) deal with the organization, budget, and staffing required to complete the project.

Q4. (Conceptual) What is a **Risk Trigger** (or Risk Symptom)?

A) The moment the project is cancelled.

B) An event or condition that warns the team that a risk is about to manifest into an issue.

C) The amount of money spent on mitigation.

D) A tool that automates testing.

> Triggers are the "early warning signs." For example, if "Developer turnover" is a risk, a "high number of sick days or low morale" might be the trigger.

Q5. (Code/Console Logic) A risk assessment script generates a priority list. Based on the logic below, which risk should be prioritized?

risk_analyzer.py

```python
def calculate_priority(prob, impact):
    return prob * impact

# Risk A: prob=0.1, impact=90
# Risk B: prob=0.6, impact=10
```

A) Risk B (Priority 6)

B) Risk A (Priority 9)

C) Both are equal.

D) Risk A is higher impact.

> Risk Exposure (Priority) = 0.1 * 90 = 9 for A, and 0.6 * 10 = 6 for B. Actually, Risk A is 9 and B is 6. Wait, A is 9. A is higher! (Correction: Risk A has exposure 9, Risk B has 6). Prioritize A.

Q6. (Scenario-Based) A team decides to "Accept" a risk that the software might be 5% slower than the requirement. What should be done next?

A) Hide the risk from the client.

B) Write more code to fix it anyway.

C) Document the risk, budget for a potential fix later, and continue monitoring.

D) Quit the project.

> Accepting a risk is a deliberate management decision. It is usually done when the cost of mitigation is higher than the impact of the risk. You must still track it.

Q7. (Comparison) What is the difference between **Risk Mitigation** and **Risk Avoidance**?

A) Avoidance is for money; Mitigation is for time.

B) Avoidance eliminates the cause of the risk; Mitigation reduces the probability or impact.

C) Avoidance is proactive; Mitigation is reactive.

D) There is no difference.

> Avoidance involves changing the project plan to eliminate the threat (e.g., not using a new, unproven language). Mitigation involves keeping the risk but taking steps to soften the blow (e.g., training developers to use the new language).

Q8. (Conceptual) In the **RMMM** plan, what do the four letters stand for?

A) Risk, Management, Mitigation, Maintenance.

B) Requirement, Metrics, Management, Monitoring.

C) Risk Mitigation, Monitoring, and Management.

D) Risk, Reliability, Metrics, Mitigation.

> RMMM stands for Risk Mitigation, Monitoring, and Management. It is a comprehensive framework used by project managers to document how risks will be handled.

Q9. (Code-Tracing) Look at this Risk Table. Which risk has the highest impact but lowest probability?

risk_table.csv

```
RiskName | Probability | Impact
-------------------------------
Cloud_Outage | 0.05        | 100
Data_Leak    | 0.02        | 90
Slow_Load    | 0.50        | 20
```

A) Cloud_Outage

B) Data_Leak

C) Slow_Load

D) All have equal risk exposure.

> Cloud_Outage probability is 0.05 and impact is 100. Data_Leak is 0.02 and impact is 90. Cloud_Outage has the highest impact (100) and a higher probability than the Data_Leak.

Q10. (Conceptual) Why is **Risk Management** sometimes referred to as "Project Insurance"?

A) Because you have to pay a bank for it.

B) Because it ensures the code is bug-free.

C) Because it identifies potential problems and prepares the team to handle them, preventing project failure.

D) Because it guarantees the project will be profitable.

> Like insurance, risk management doesn't stop bad things from happening, but it prepares the organization financially and operationally to survive them if they do happen.

Deep Dive: The Risk Assessment Matrix

Managers use a 2x2 matrix to categorize risks:

- **High Prob / High Impact:** "Showstoppers." Must be avoided or mitigated aggressively.
- **Low Prob / High Impact:** "Catastrophes." Requires a robust contingency plan.
- **High Prob / Low Impact:** "Nuisances." Should be mitigated with low-cost efforts.
- **Low Prob / Low Impact:** "Trivialities." Can usually be accepted.

Stable Web Resources

- Wikipedia: Project Management Triangle
- Project Management Institute (PMI)

Quick Revision Cheat Sheet

- **Risk Exposure:** Prob * Impact.
- **Avoidance:** Change plan to eliminate risk.
- **Mitigation:** Reduce likelihood or effect.
- **Transfer:** Buy insurance / Outsourcing.
- **Acceptance:** Budget for it, monitor it, deal with it if it happens.
- **Risk Trigger:** Warning sign.
- **Issue:** A risk that already happened.

END OF SUBTOPIC 15 — MOVING TO SUBTOPIC 16

Subtopic 16: Software Security Engineering (10 MCQs)

Quick Notes

- **Security Engineering:** Building systems that remain dependable in the face of malicious intent (attackers).
- **CIA Triad:**
  - *Confidentiality:* Only authorized users can read data.
  - *Integrity:* Data is accurate, complete, and protected from unauthorized modification.
  - *Availability:* Data and services are accessible when needed.

- **Threat Modeling:** A structured approach to identify threats (e.g., STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).
- **Least Privilege:** Users/services should have only the permissions necessary for their task.
- **Secure by Design:** Security is not an "add-on" at the end; it is integrated into requirements, design, and coding.

Common Traps & Confusions

- **Security vs. Safety:** *Security* protects against malicious intent; *Safety* protects against accidents or system failure.
- **The "Human Factor":** The most secure system can be compromised by a simple phishing attack. Social engineering is often a bigger threat than code vulnerabilities.
- **Encryption ≠ Security:** Encrypting data is useless if the encryption keys are stored in cleartext in the source code.

MCQs

Q1. (Scenario-Based) An application stores all user passwords in the database using the `MD5` hashing algorithm. A security auditor flags this as a critical vulnerability. What is the primary reason?

A) MD5 is too slow for modern databases.

B) MD5 is cryptographically broken and prone to collision attacks and fast brute-forcing.

C) MD5 does not allow for password recovery.

D) MD5 only works on numeric data.

> MD5 is extremely fast, which is a major weakness for password hashing because attackers can guess millions of passwords per second. Modern systems should use slow, salted hashes like Argon2 or bcrypt.

Q2. (Conceptual) In the **STRIDE** threat model, what does **"Repudiation"** refer to?

A) The attacker steals the user's password.

B) A user performing an action (e.g., a financial transfer) and then denying they did

it because there is no proof.

C) The server crashing due to a DDoS attack.

D) Unauthorized access to private customer files.

> Repudiation threats occur when a system lacks non-repudiation features (like digital signatures or audit logs), allowing a malicious user to perform an action and later claim "I didn't do it."

Q3. (Scenario-Based) An attacker exploits a vulnerability in a web application to force the database to return records from other users' accounts. This is a classic violation of which part of the CIA Triad?

A) Integrity

B) Confidentiality

C) Availability

D) Non-repudiation

> Confidentiality ensures that only authorized parties can access data. Exposing private user records to an unauthorized person is a breach of confidentiality.

Q4. (Code/Console Logic) You are auditing a piece of C code. Which function call is flagged as dangerous in modern security standards?

audit_source.c

```c
char buffer[10];
gets(buffer); // Reads user input into buffer
```

A) `printf()`

B) `malloc()`

C) `gets()`

D) `strlen()`

> The `gets()` function in C does not check the size of the buffer. If the user provides more than 10 characters, it causes a "Buffer Overflow," which can lead to memory corruption or arbitrary code execution.

Q5. (Conceptual) What is the purpose of "Salting" in password storage?

A) To make the password longer.

B) To append a unique, random string to each password before hashing, preventing Rainbow Table attacks.

C) To make the password easier to type.

D) To encrypt the password with a public key.

> Rainbow tables are pre-computed lists of hashes. Salting ensures that even if two users have the same password, their stored hashes will be completely different, rendering rainbow tables ineffective.

Q6. (Scenario-Based) A developer sets up a new cloud server but leaves the default admin username and password (e.g., `admin/admin`). This is a failure in:

A) Cryptography

B) Denial of Service mitigation

C) Authentication and Access Control

D) Input Validation

> Authentication is the process of verifying identity. Using default, well-known credentials bypasses authentication entirely, making the system trivial to compromise.

Q7. (Comparison) What is the difference between a **Vulnerability** and a **Threat**?

A) Vulnerabilities are intentional; Threats are accidental.

B) A Vulnerability is a weakness in the system; a Threat is a potential actor or event that could exploit that weakness.

C) Threats only apply to software; Vulnerabilities only apply to hardware.

D) They mean the same thing.

> You have a vulnerability (e.g., an unpatched login page) and a threat (e.g., a hacker who wants to steal your data). Security is the process of fixing the vulnerability before the threat can exploit it.

Q8. (Scenario-Based) A company implements a policy where an employee must provide both a password and a physical security token to access the corporate server. This is called:

A) Single Sign-On (SSO)

B) Multi-Factor Authentication (MFA)

C) Authorization

D) Role-Based Access Control

> MFA requires at least two of the three factors: something you know (password), something you have (token), or something you are (biometrics).

Q9. (Conceptual) What is the main objective of a **Penetration Test (Pen Test)**?

A) To write documentation for the system.

B) To train new employees.

C) To simulate a real-world attack to identify security weaknesses before malicious actors find them.

D) To backup the database.

> Penetration testing is "White Hat" hacking. It's a proactive measure to verify security by trying to break the system from the outside.

Q10. (Scenario-Based) A web application is suffering from a **"Cross-Site Scripting (XSS)"** attack. What is actually happening?

A) The attacker is sending too many requests to crash the server.

B) The attacker is injecting malicious JavaScript into the webpage, which executes in the browser of other users.

C) The attacker is directly accessing the database via SQL.

D) The attacker is stealing the physical server.

> XSS targets the end-user. If the application doesn't sanitize input, an attacker can post a comment containing `<script>stealCookies();</script>`, which then executes in the browser of anyone who views that comment.

Deep Dive: The Defense in Depth Strategy

Security is never solved by a single "magic bullet." The **Defense in Depth** strategy assumes that any single security layer will eventually fail, so you must have multiple, overlapping layers:

- **Outer Layer:** Network Firewalls/WAF to block malicious traffic.
- **Application Layer:** Input validation, prepared statements (XSS/SQLi defense).

- **Authentication Layer:** MFA, strong password policies.
- **Data Layer:** Encryption at rest, row-level security in the database.
- **Audit Layer:** Monitoring, logging, and incident response teams.

If the firewall fails, the application layer catches the exploit. If the application layer is bypassed, the database encryption protects the data. This "layered" approach is what prevents simple bugs from becoming catastrophic data breaches.

Stable Web Resources

- OWASP: Top 10 Security Risks
- Wikipedia: Information Security

Quick Revision Cheat Sheet

- **CIA Triad:** Confidentiality, Integrity, Availability.
- **Authentication:** Who are you? **Authorization:** What can you do?
- **SQLi:** Use Parameterized Queries.
- **XSS:** Sanitize all output.
- **Salting:** Random bits added to password hashes.
- **Least Privilege:** Minimum access possible.